



Deep Learning

Dr. Monali Mavani

BITS Pilani

Pilani Campus

Credits: Slides are adopted from Stanford CS224N: Natural Language Processing with Deep Learning and many others who made their course materials freely available online

Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Content



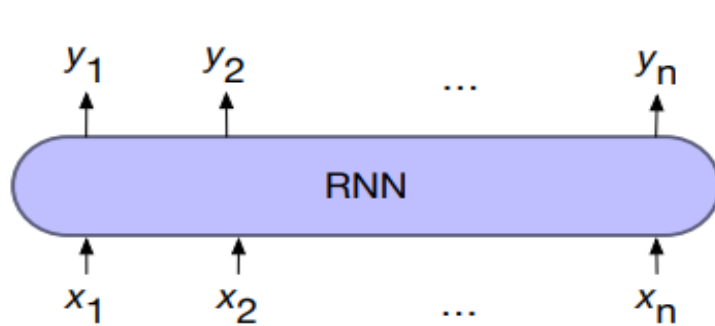
- Encoder-Decoder with RNN
- Issues with recurrent models
- Attention mechanism
- Transformer architecture
- Transformers for NLP
- Transformers for computer vision

Deep Learning Architectures for Sequence Processing



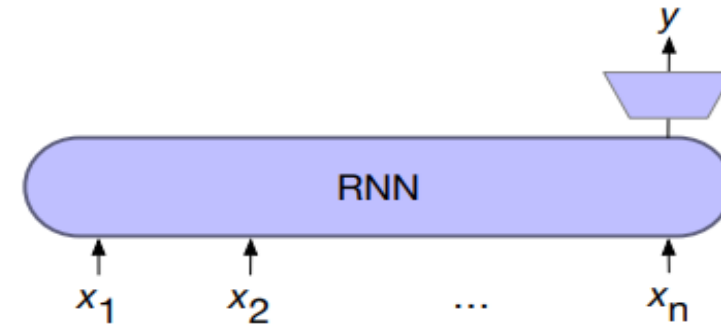
- **Recurrent neural networks and transformer** networks
- Both capture and exploit the **temporal** nature of language
- use the **prior context**, allowing the model's decision to depend on information from words in the past.
- **The transformer uses mechanisms (self-attention and positional encodings) that help focus on how words relate to each other over long distances**

RNN Architectures for NLP Tasks



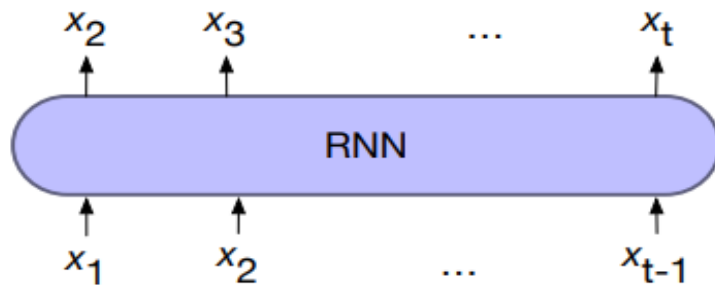
a) sequence labeling

Ex: POS Tagging, Named Entity Tagging



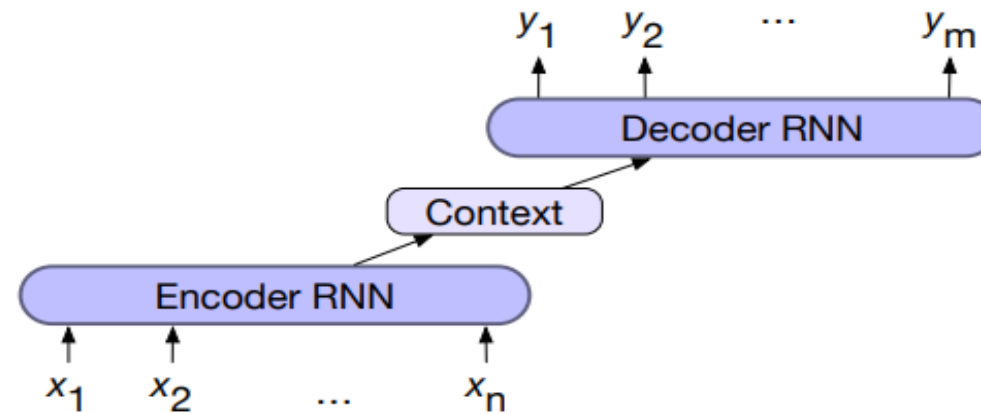
b) sequence classification

Ex: Sentiment Analysis



c) language modeling

Ex: Predict Next Word



d) encoder-decoder

Ex: Language Translation

Encoder-Decoder or Sequence-to-Sequence Networks



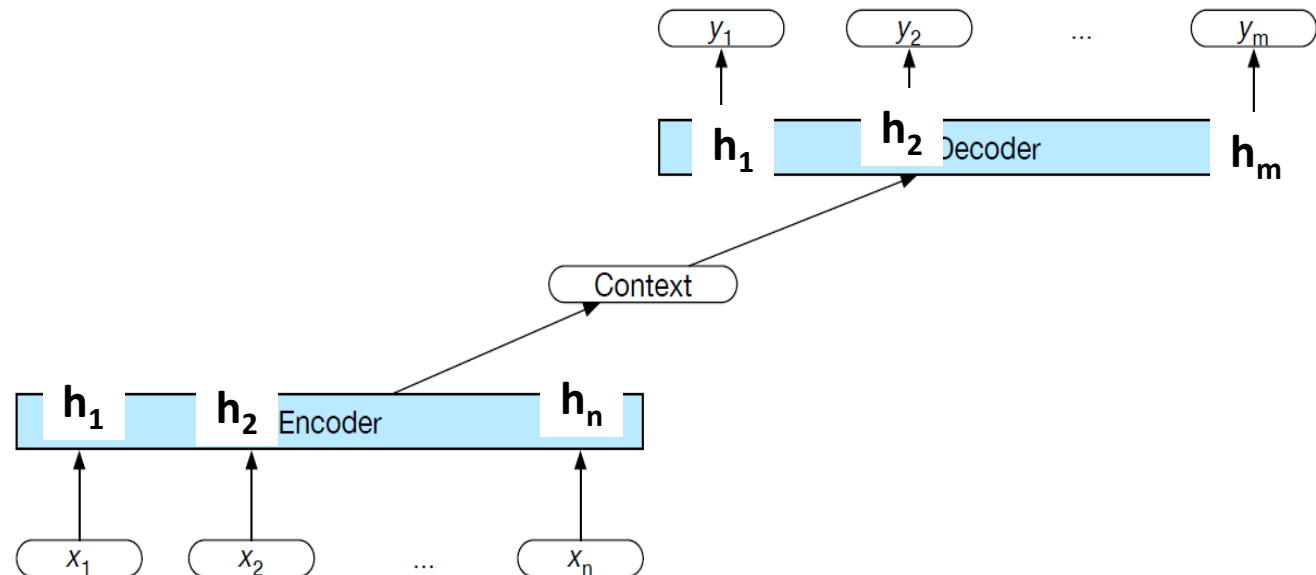
- **Models capable of generating contextually appropriate, arbitrary length, output sequences**
- One neural network takes input and produces a neural representation
- Another network produces output based on that neural representation
- Many NLP tasks can be phrased as sequence-to-sequence:
 - Summarization (long text → short text)
 - Dialogue (previous utterances → next utterance)
 - Parsing (input text → output parse as sequence)
 - Code generation (natural language → Python code)

Encoder-Decoder architecture

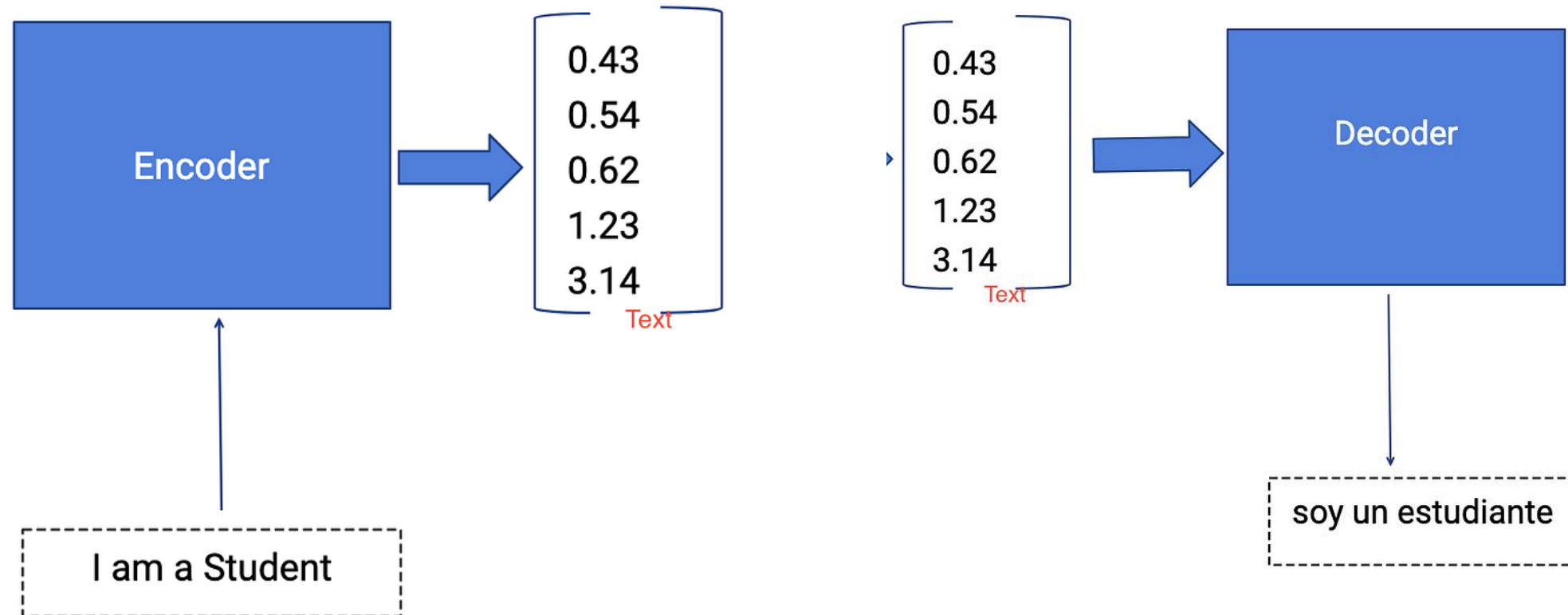


1. **Encoder:** accepts an input sequence, $\mathbf{x}_{1:n}$ and generates a corresponding sequence of contextualized representations, $\mathbf{h}_{1:n}$
2. **Context vector $\mathbf{c}$:** function of $\mathbf{h}_{1:n}$ and conveys the essence of the input to the decoder.
3. **Decoder:** accepts $\mathbf{c}$ as input and generates an arbitrary length sequence of hidden states $\mathbf{h}_{1:m}$ from which a corresponding sequence of output states $\mathbf{y}_{1:m}$ can be obtained.

LSTMs, convolutional networks, and Transformers can all be employed as encoders/decoders



Simple MT example using encoder-decoder



MT with RNN based encoder-decoder

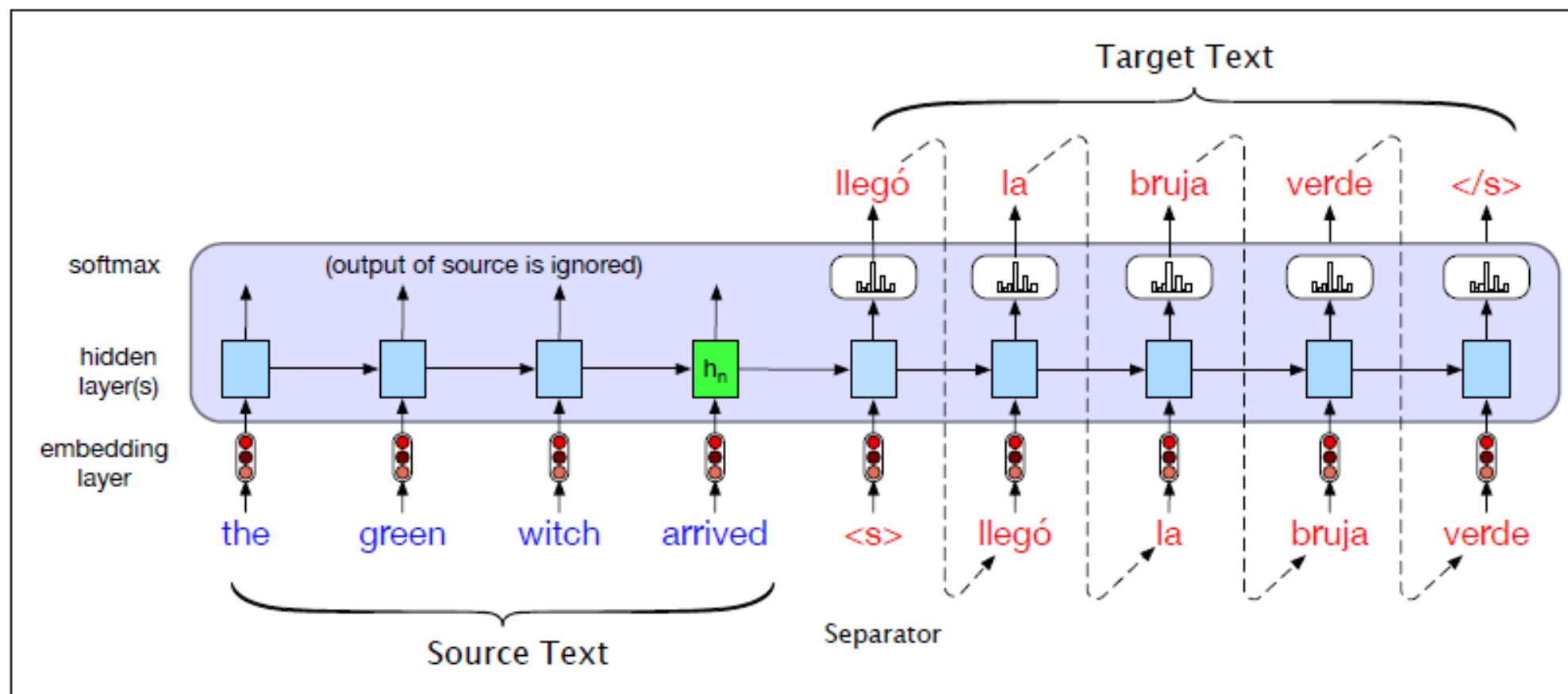
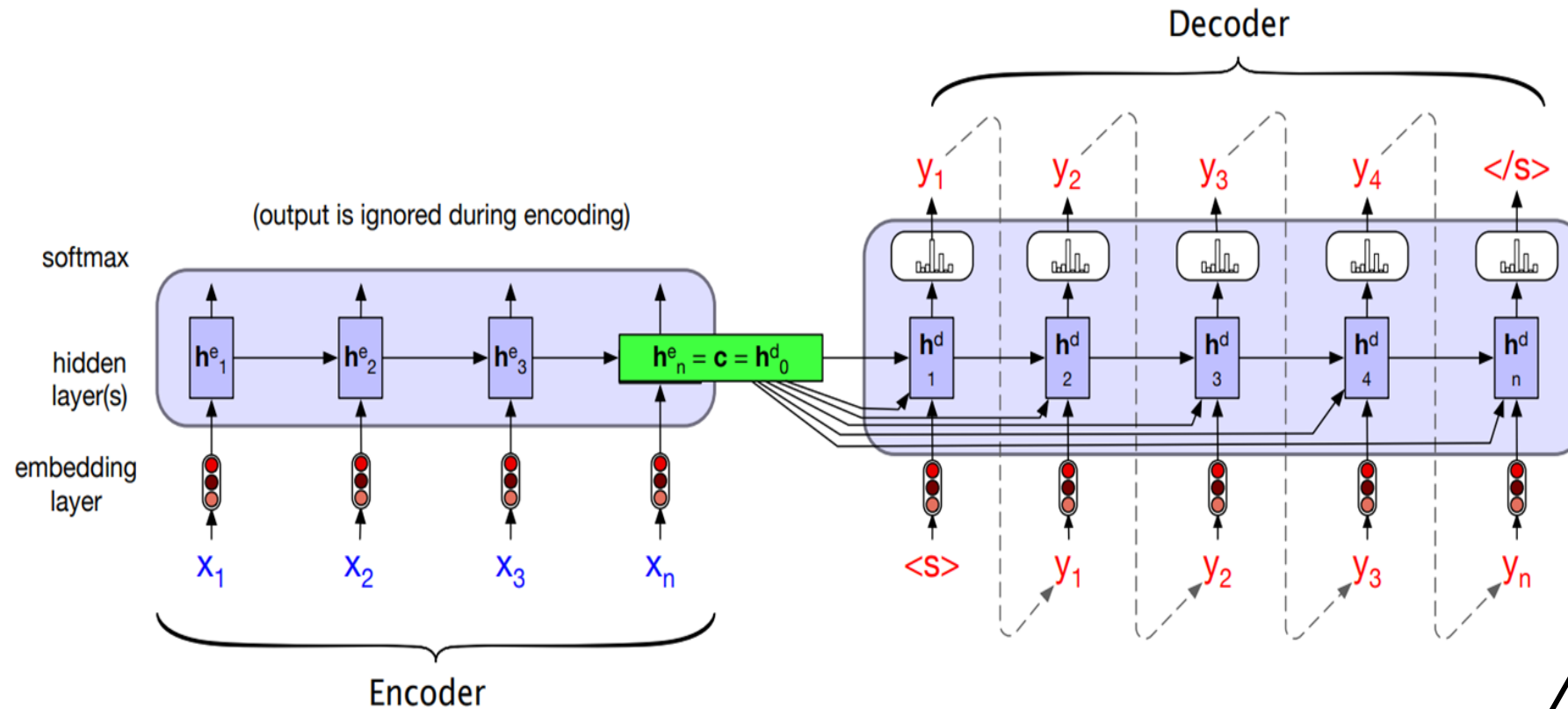


Figure 10.4 Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

Encoder - Decoder using RNN



Encoder - Decoder for Language Translation



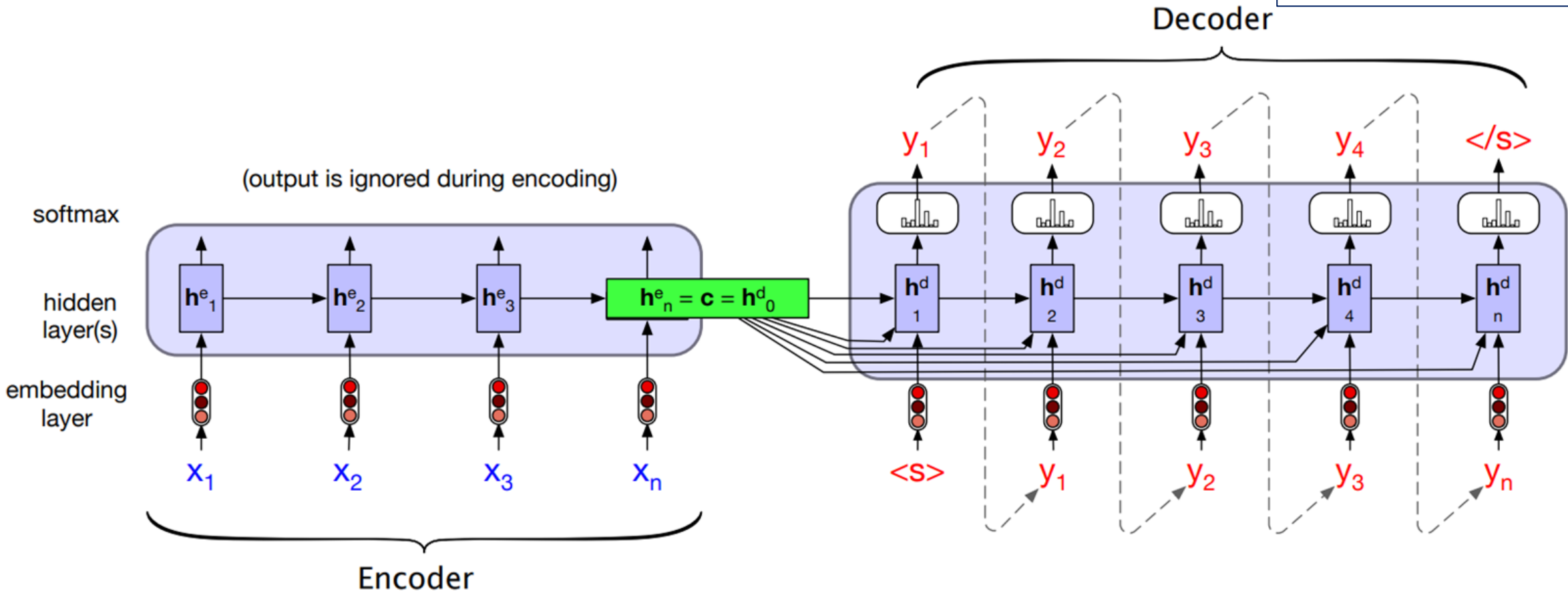
- Encoder generates a contextualized representation of the input (last state)
- Decoder takes that state and autoregressively generates a sequence of outputs

word generated at each time step is conditioned on word from previous step.

Encoder - Decoder

Encoder - Decoder for Language Translation

$$\begin{aligned} \mathbf{c} &= \mathbf{h}_n^e \\ \mathbf{h}_0^d &= \mathbf{c} \\ \mathbf{h}_t^d &= g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\ \mathbf{z}_t &= f(\mathbf{h}_t^d) \\ y_t &= \text{softmax}(\mathbf{z}_t) \end{aligned}$$



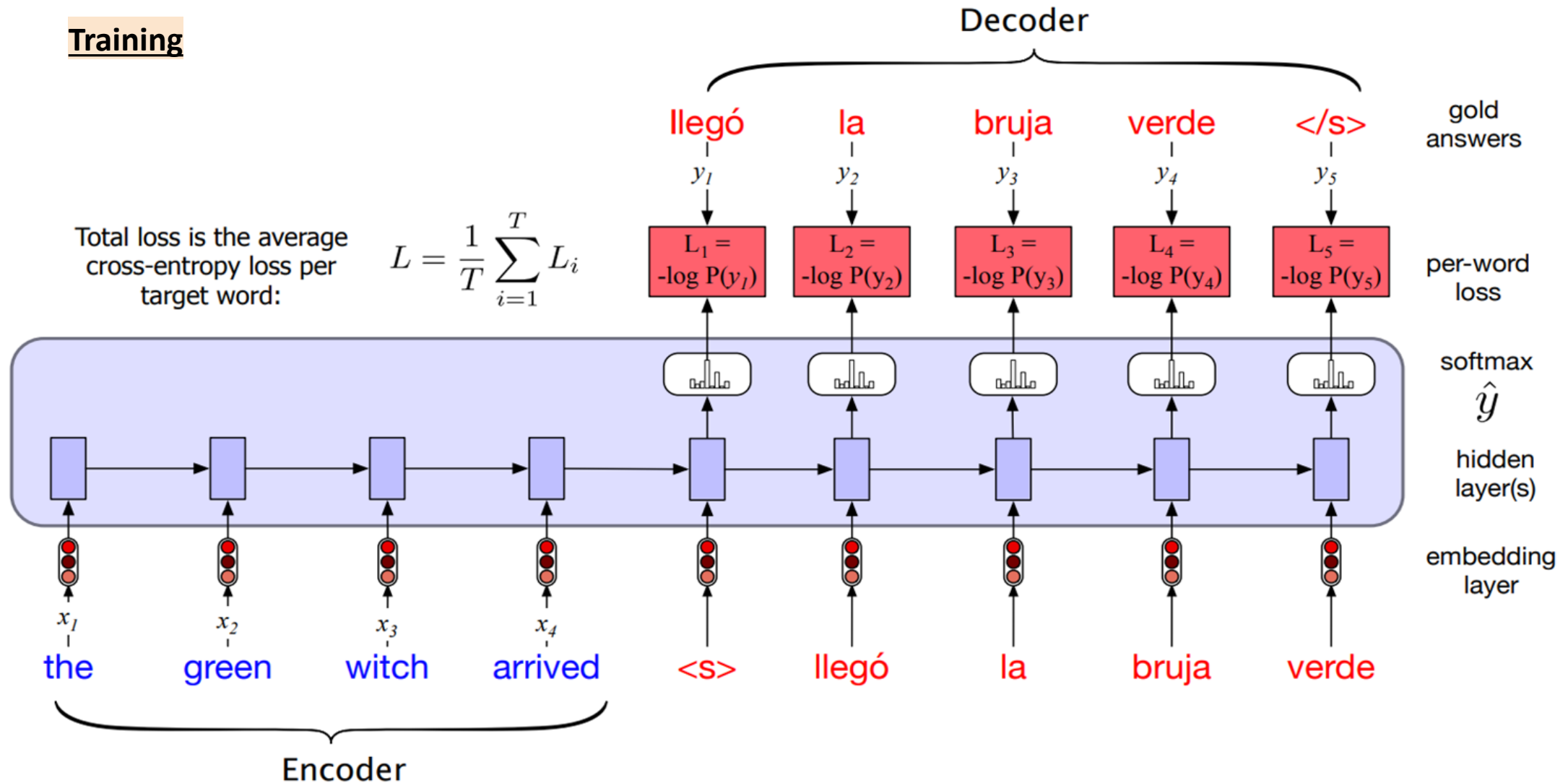
Encoder - Decoder



Training

Total loss is the average cross-entropy loss per target word:

$$L = \frac{1}{T} \sum_{i=1}^T L_i$$



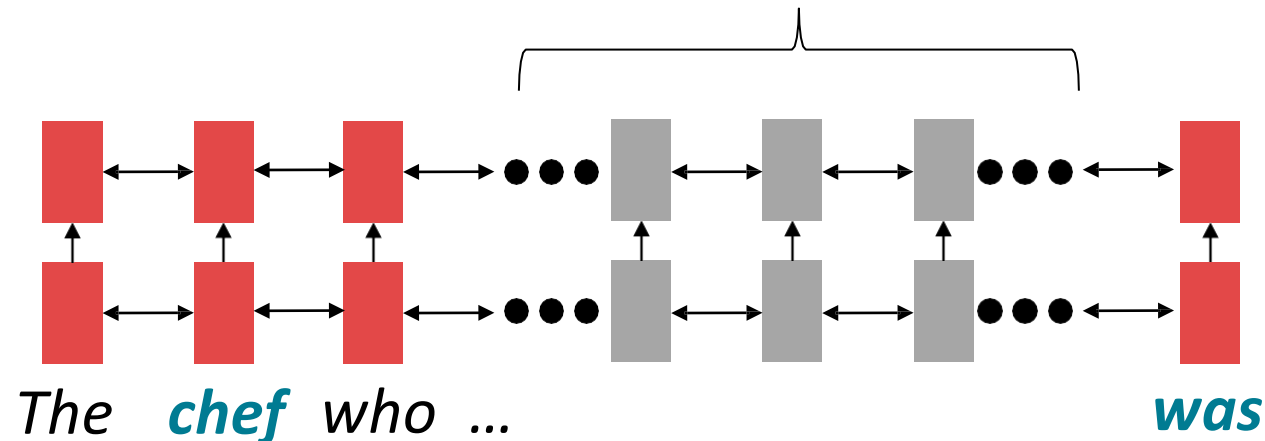
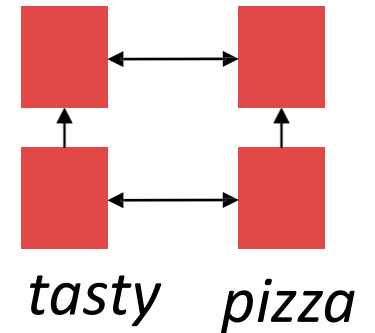
Teaching Forcing

- Force the system to use the gold target token from training as the next input x_{t+1} , rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{y}_t$.
- Speeds up training

Issues with recurrent models



- **$O(\text{sequence length})$** steps for distant word pairs to interact means:
 - Forward and backward passes have **$O(\text{sequence length})$** unparallelizable operations
- RNNs are unrolled “left-to-right”.
- This encodes linear locality: a useful heuristic!
 - Nearby words often affect each other’s meanings
 - Hard to learn long-distance dependencies (because gradient problems!)

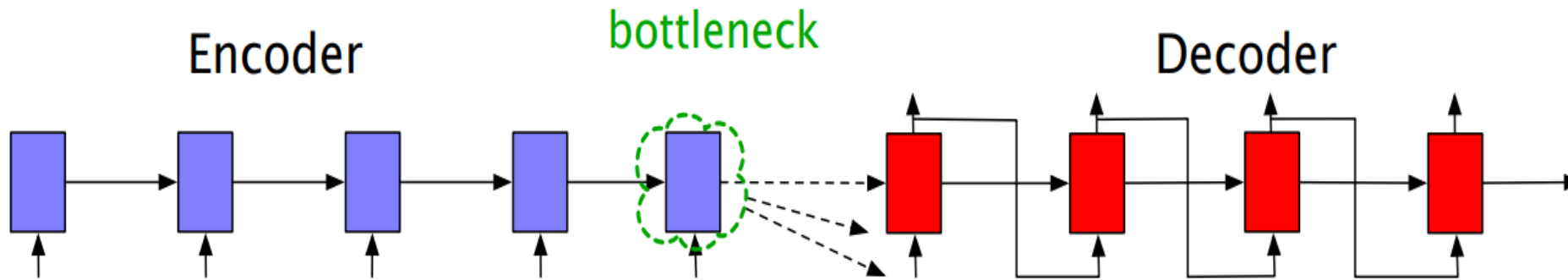


Effect of vanishing gradient on RNN-LM



- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- To learn from this training example, the RNN-LM needs to **model the dependency** between “tickets” on the 7th step and the target word “tickets” at the end.
- But if the gradient is small, the model **can't learn this dependency**
 - So, the model is **unable to predict similar long-distance dependencies** at test time
- In practice a simple RNN will only condition ~7 tokens back [**vague rule-of-thumb**]

Encoder-decoder bottleneck

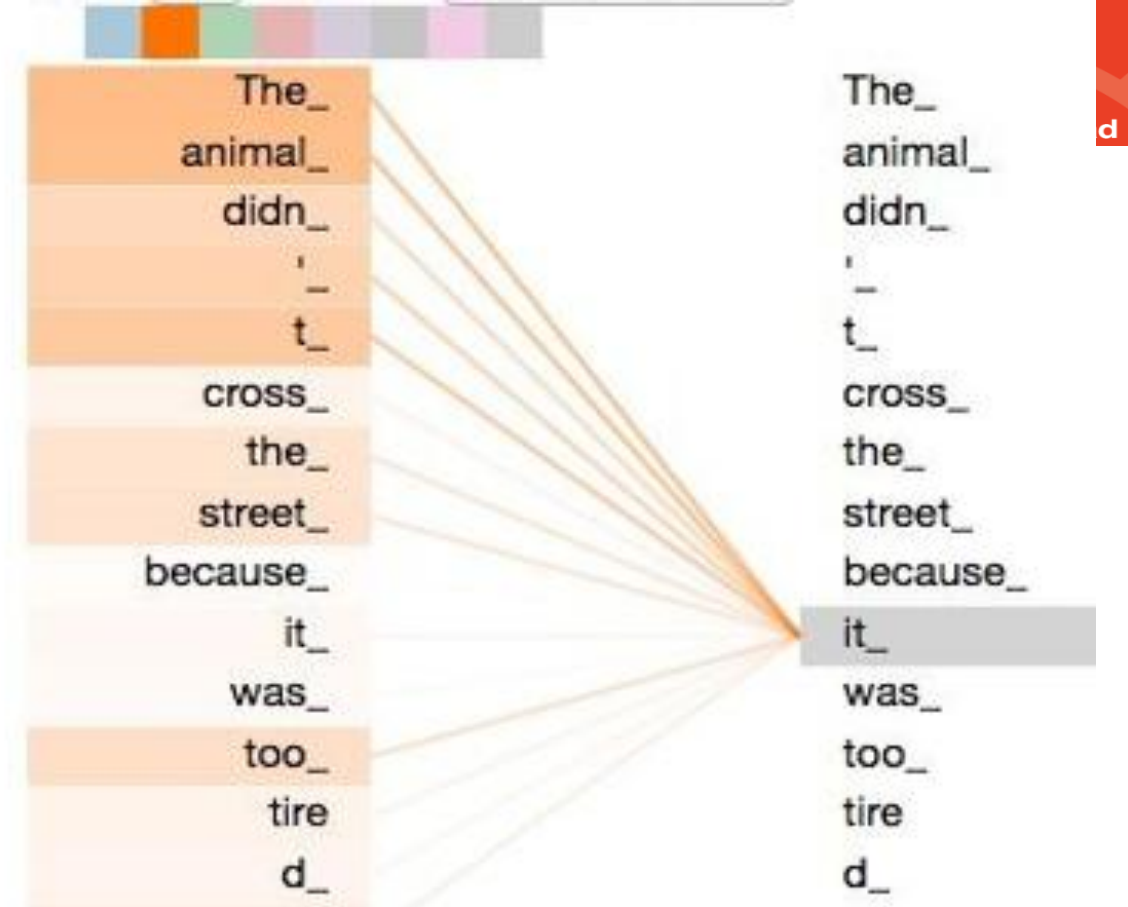


- Final state of the **E** is the only context available to **D**
- It must represent absolutely everything about the meaning of the source text
- The only thing the decoder knows about the source text is what's in this context vector

Context

“The animal didn't cross the street because it was too tired”

What is “it”?

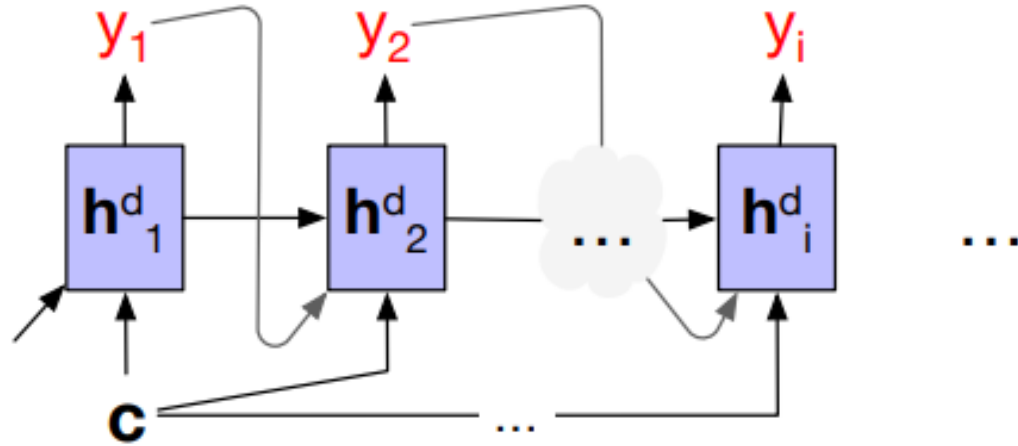


She is eating a green apple.

high attention

low attention

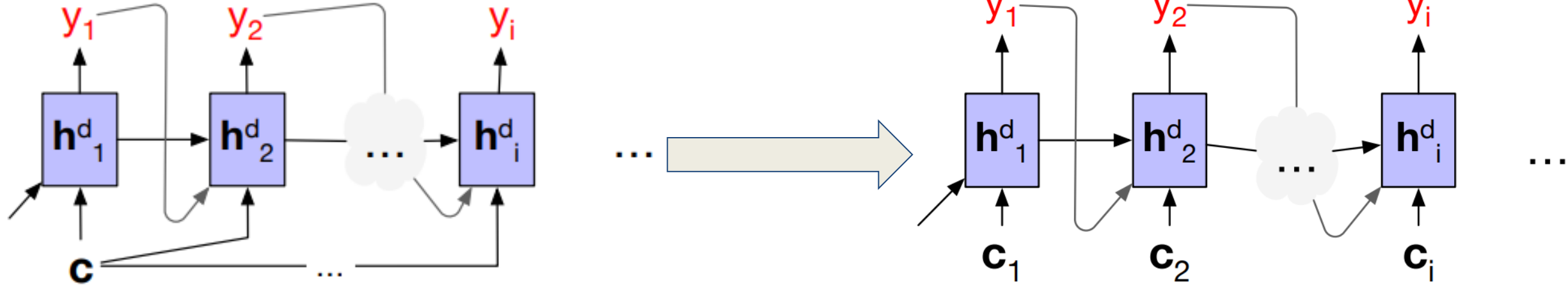
Attention ! (in RNN based encoder-decoder)



Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

Attention ! (in RNN based encoder-decoder)



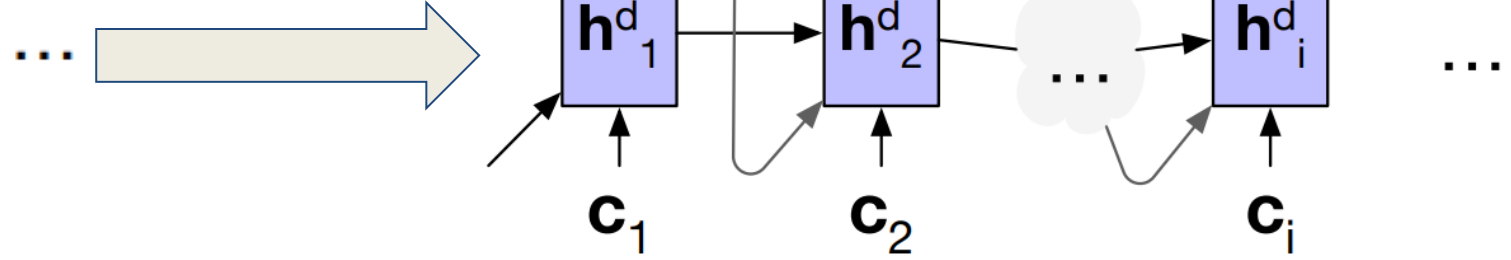
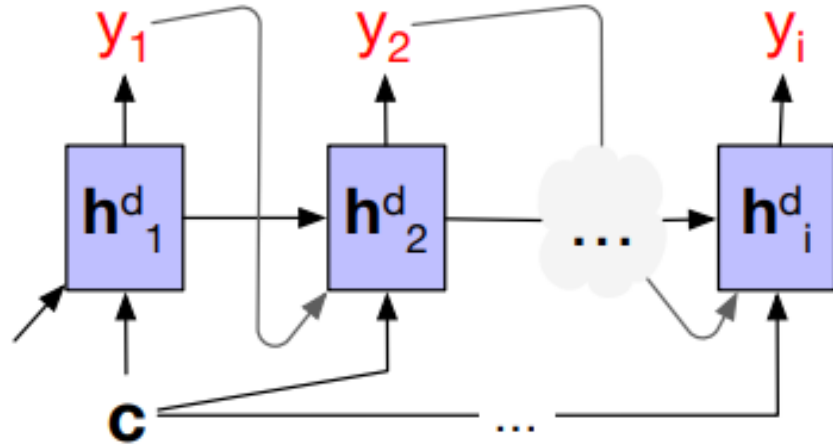
Without attention, a decoder sees the same context vector, which is a static function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

Attention !



Without attention, a decoder sees the same context vector, which is a static function of all the encoder hidden states

With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

With attention, decoder gets information from all the hidden states of the encoder, not just the last hidden state of the encoder

Each context vector is obtained by taking a weighted sum of all the encoder hidden states.

The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing

Attention !



Step -1 : Find out how relevant each encoder state is to the present decoder state $\mathbf{h}_{i-1}^d$

Compute a score of similarity between $\mathbf{h}_{i-1}^d$ and all the encoder states : $score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)$

Dot Product Attention : $score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$

Step -2 : Normalize all the scores with softmax to create a vector of weights, $\alpha_{i,j}$

$\alpha_{i,j}$ indicates the proportional relevance of each encoder hidden state j to the prior hidden decoder state, $\mathbf{h}_{i-1}^d$

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \quad \forall j \in e) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}\end{aligned}$$

Attention !



Step -3 : Given the distribution in α , compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

Plus : In step-1, we can get a more powerful scoring function by parameterizing the score with its own set of weights, $\mathbf{W}_s$:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \mathbf{W}_s \mathbf{h}_j^e$$

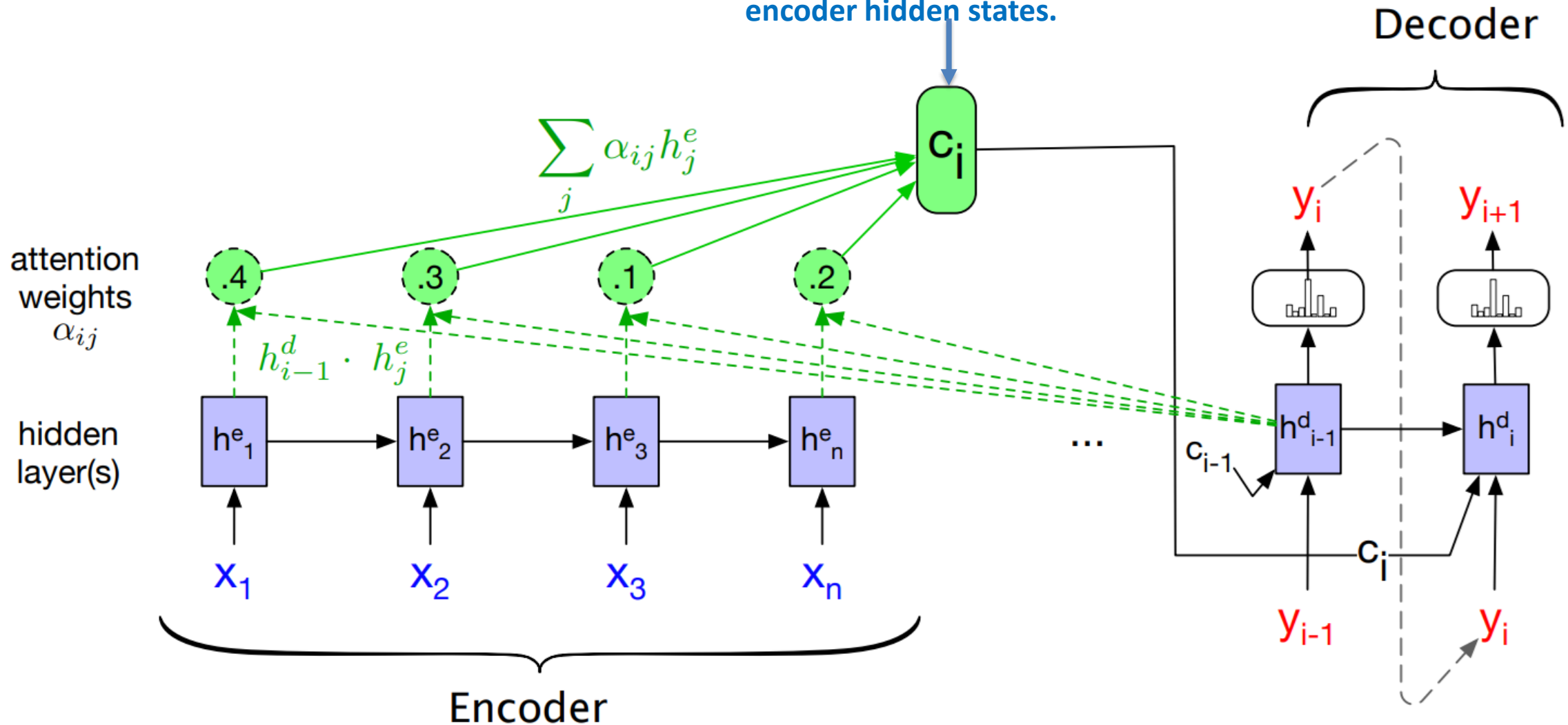
$\mathbf{W}_s$, is trained during normal end-to-end training,
 $\mathbf{W}_s$, gives the network the ability to learn which aspects of similarity between the decoder and encoder states are important to the current application.

Attention !



compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states.

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



Sequence-to-Sequence with RNNs and Attention

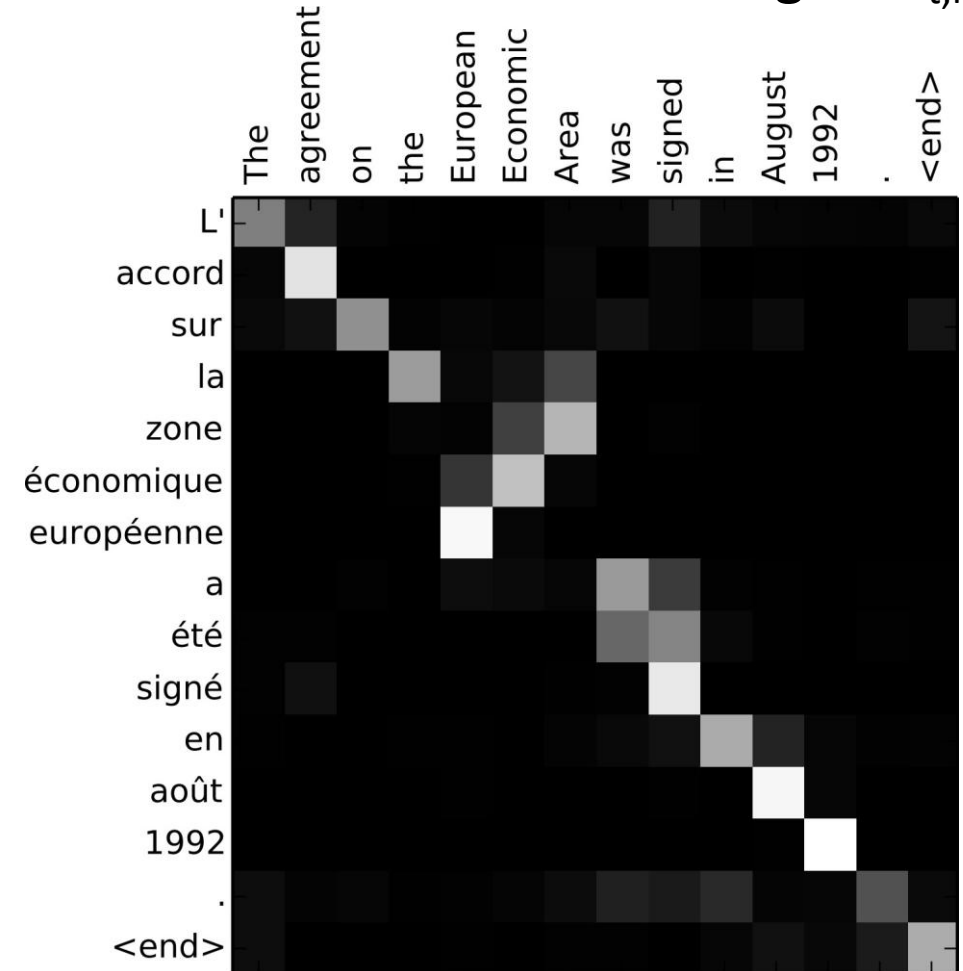


Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Visualize attention weights $a_{t,i}$



Sequence-to-Sequence with RNNs and Attention

Visualize attention weights $a_{t,i}$

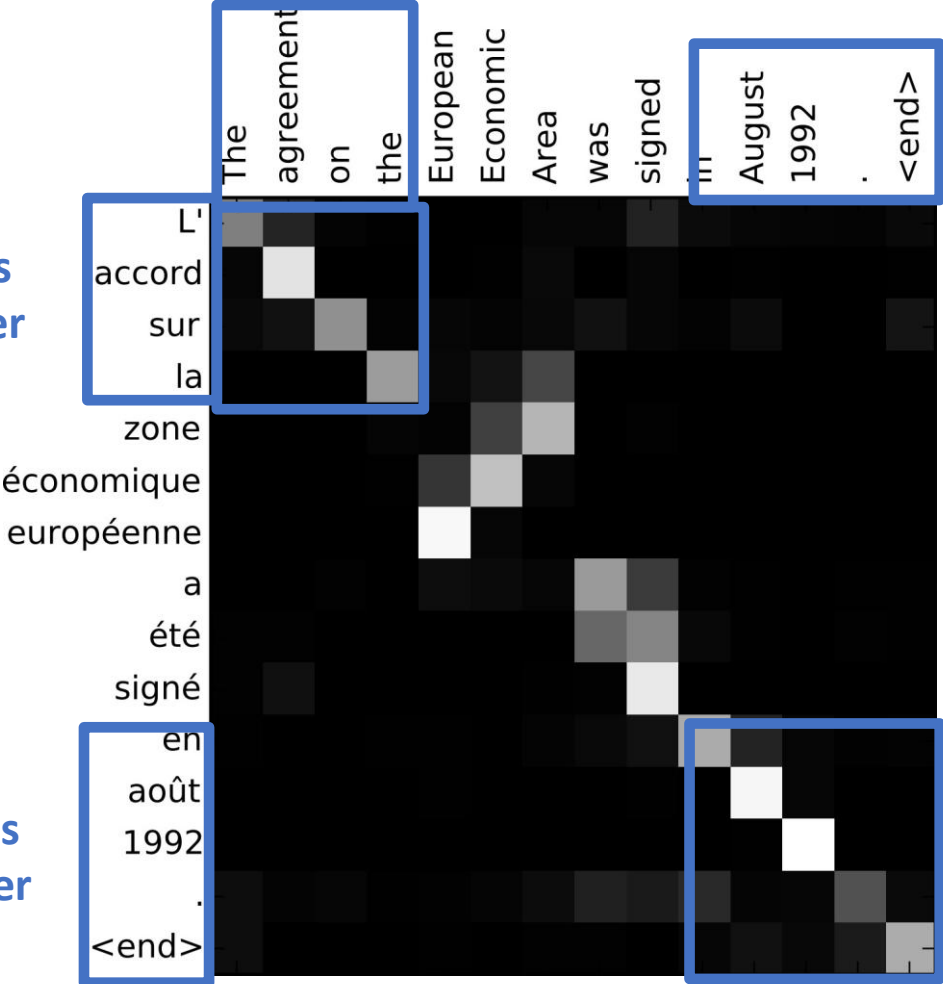
Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order



Sequence-to-Sequence with RNNs and Attention

Visualize attention weights $a_{t,i}$

Example: English to French translation

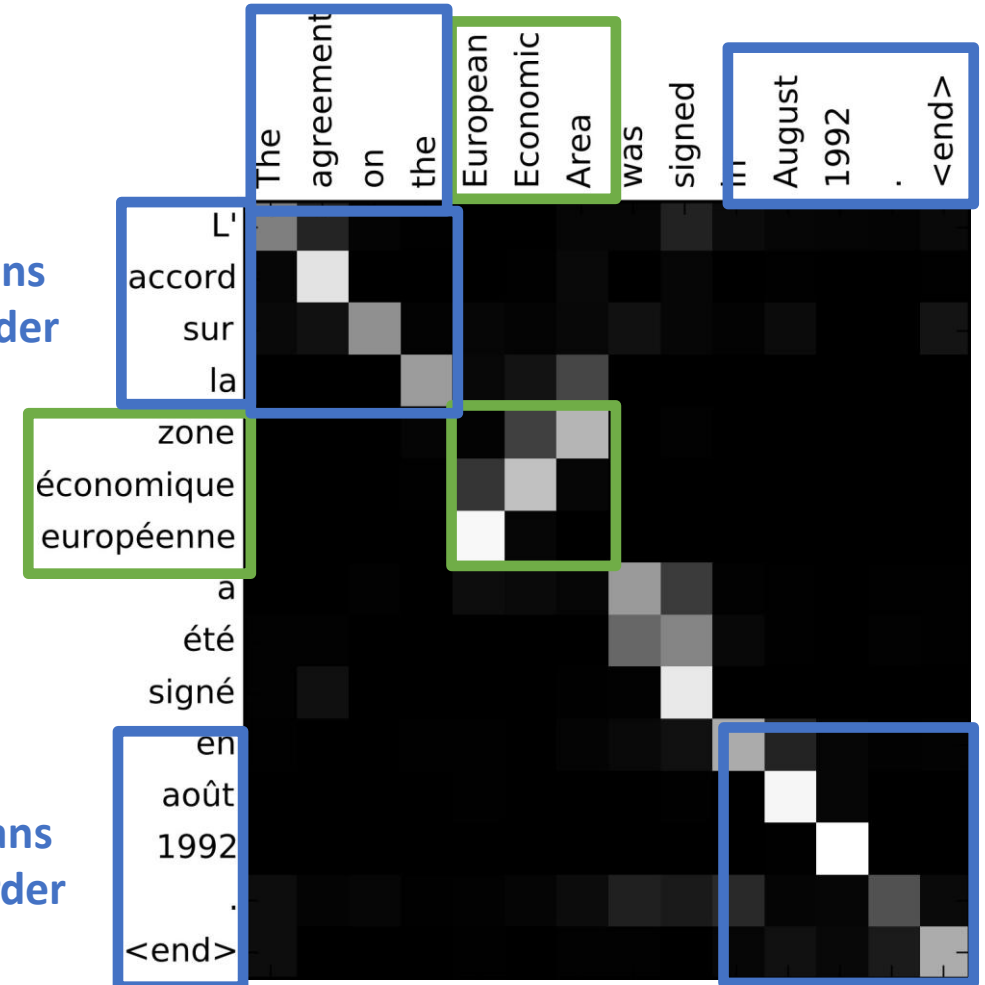
Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

Diagonal attention means words correspond in order



Sequence-to-Sequence with RNNs and Attention



Visualize attention weights $a_{t,i}$

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

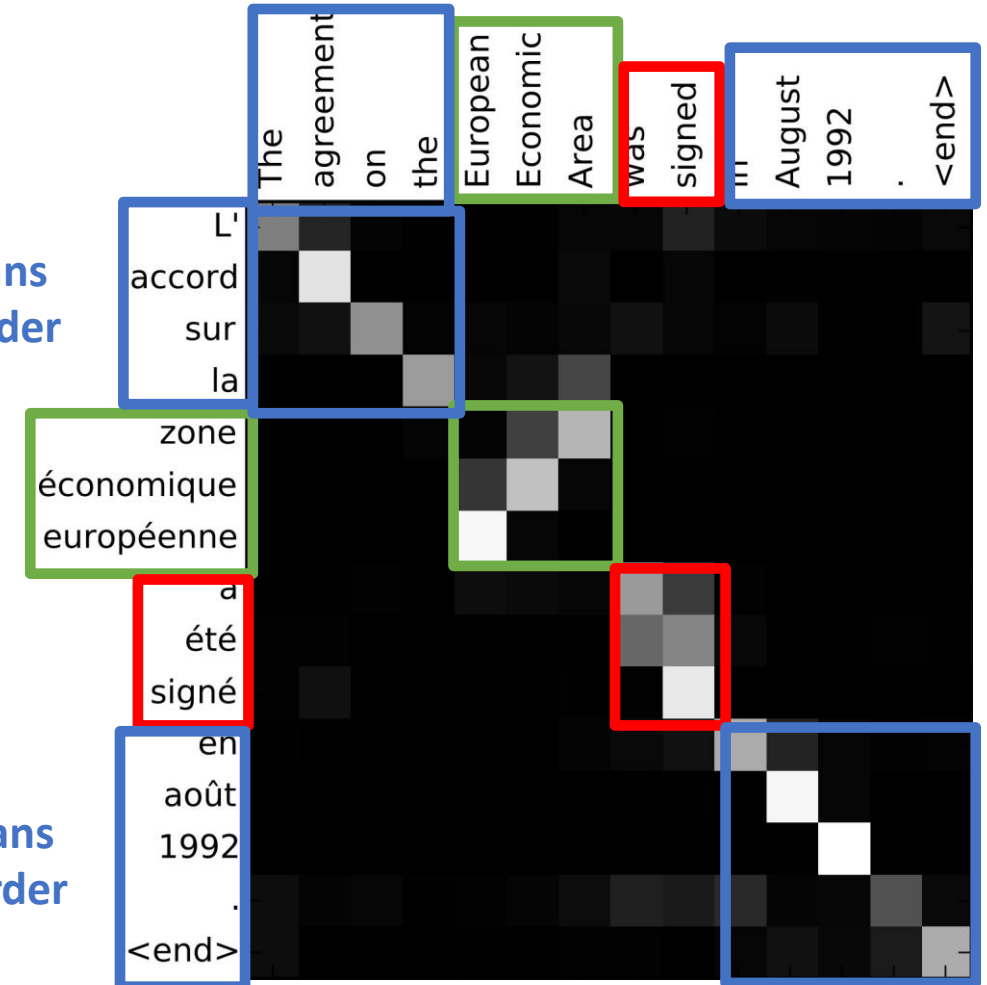
Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

Verb conjugation

Diagonal attention means words correspond in order



Attention is a *general* Deep Learning technique

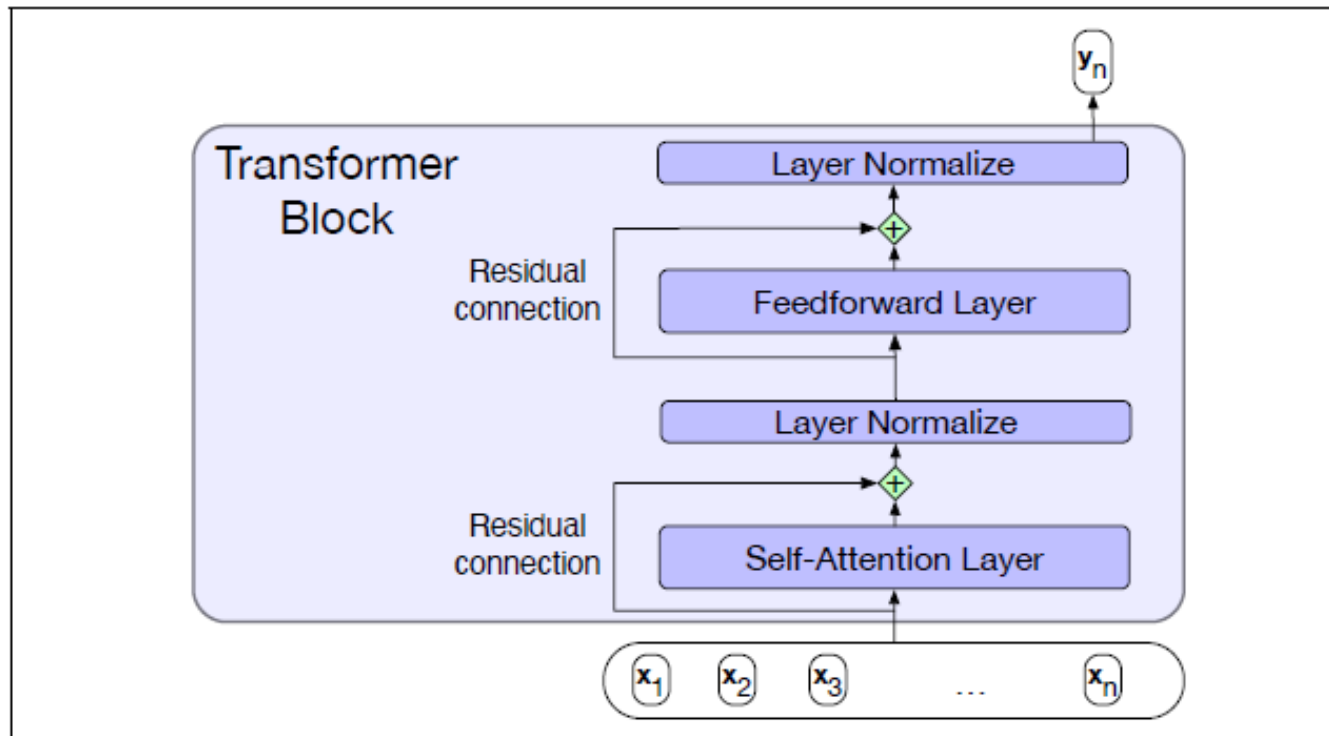


- Attention significantly **improves performance**
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention **solves the bottleneck problem**
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention **helps with the vanishing gradient problem**
 - Provides shortcut to faraway states
- By inspecting attention distribution, we see what the decoder was focusing on
- **Attention** $\Rightarrow$ Ability to compare an item of interest to a collection of other items in a way that reveals their relevance in the current context.
- **Self-attention** $\Rightarrow$
 - > Set of **comparisons** are to other elements ***within a given sequence***
 - > Use these comparisons to compute an output for the current input

Transformers



- 2017, NIPS, Vaswani et. al., **Attention Is All You Need !!!**
- Made up of **transformer blocks** in which the key component is **self-attention** layers
- Transformers are **not based on recurrent connections** $\Rightarrow$ Parallel implementations possible $\Rightarrow$ Efficient to scale (comparing LSTM)



- Each Block consists of:
 - Self-attention
 - Add & Norm
 - Feed-Forward
 - Add & Norm

Figure 9.18 A transformer block showing all the layers.



Tokenization

Tokenization



Tokenization: Turning words into tokens

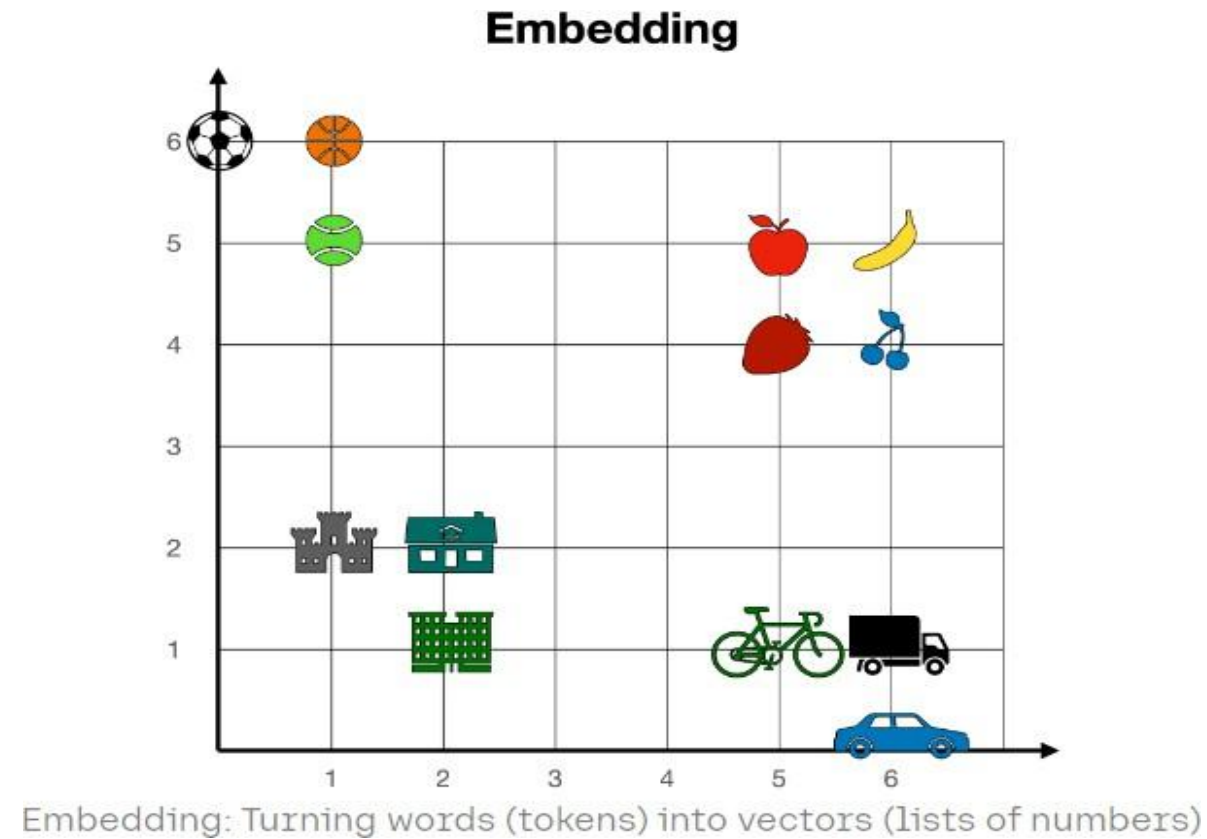
For example, if the sentence is “Write a story”, then the 4 corresponding tokens will be <Write>, <a>, <story>, and <.>.

Transformers

Input



Embedding



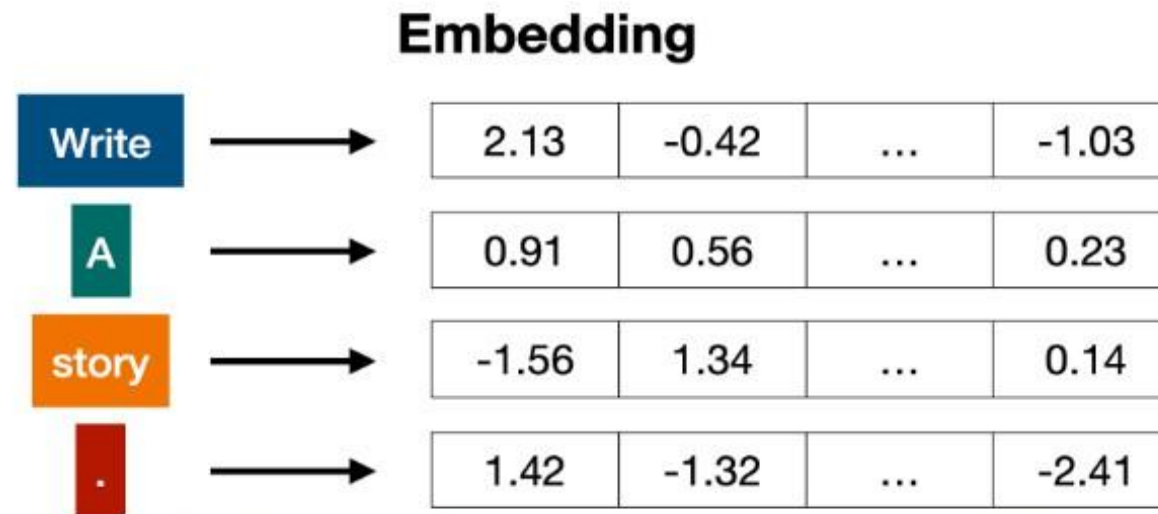
Transformers

Input



Embedding

For example, if the sentence we are considering is “Write a story.” and the tokens are <Write>, <a>, <story>, and <.>, then each one of these will be sent to a long vector, and we’ll have four vectors.



In general embeddings send every word (token) to a long list of numbers

Transformers

Self attention layer

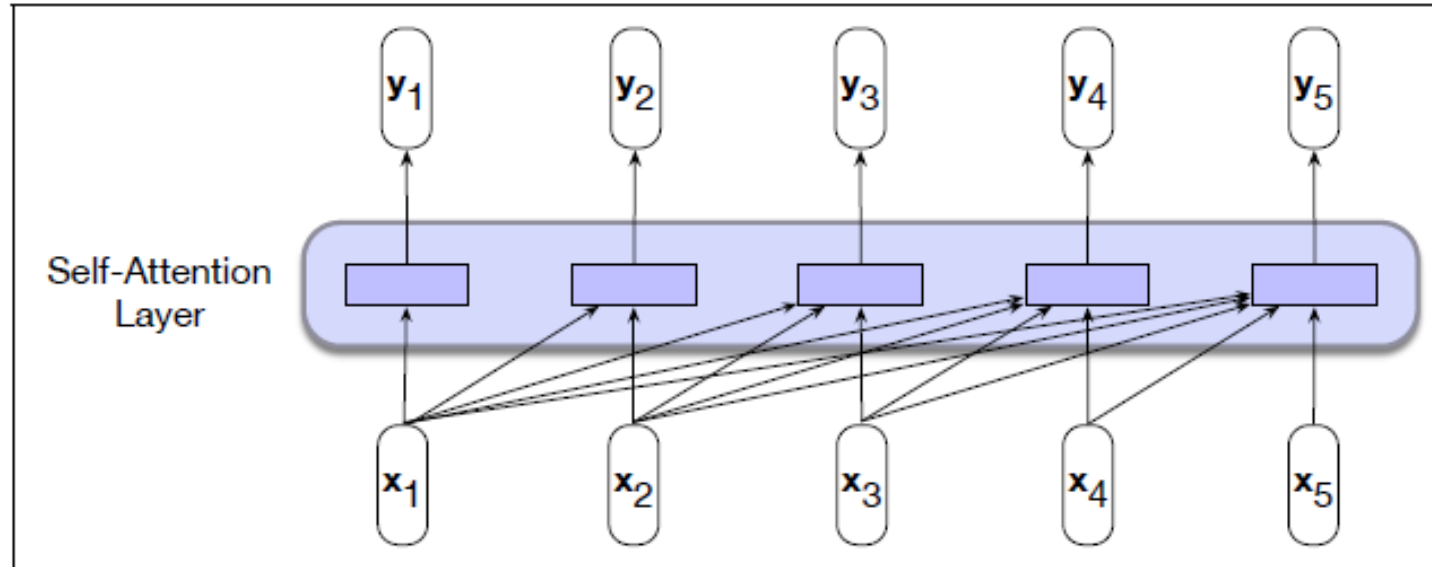
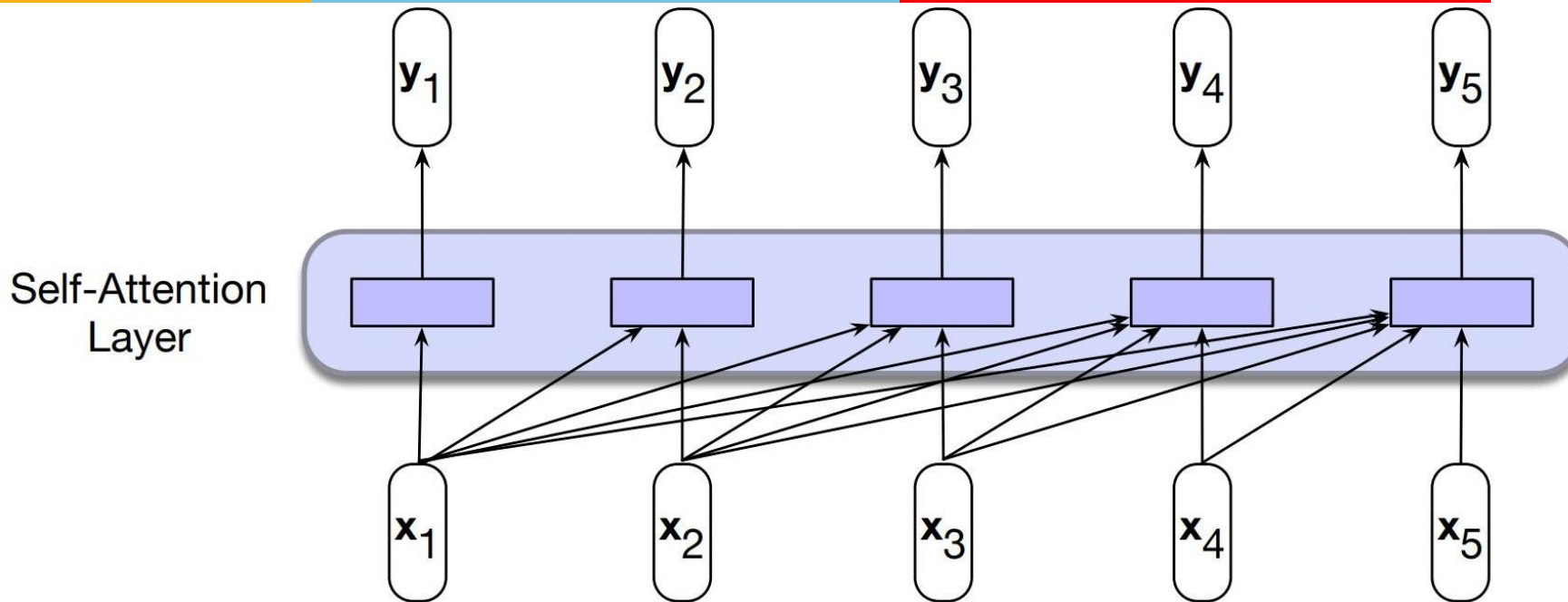


Figure 9.15 Information flow in a causal (or masked) self-attention model. In processing each element of the sequence, the model attends to all the inputs up to, and including, the current one. Unlike RNNs, the computations at each time step are independent of all the other steps and therefore can be performed in parallel.

- Computation of y_3 is based on a set of comparisons between the input x_3 and its preceding elements x_1 and x_2 , and to x_3 itself
- When processing each item in the input, the model has no access to information about inputs beyond the current one.
- This ensures that we can use this approach to create language models and use them for **autoregressive generation**

Self-Attention | Transformers



$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

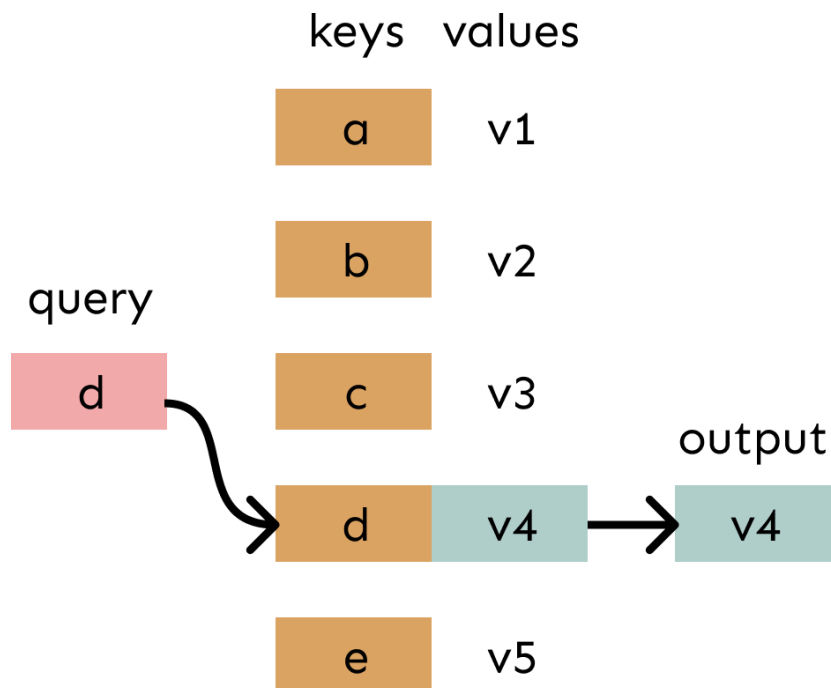
$$\begin{aligned} \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^i \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \leq i \end{aligned}$$

Attention as a soft, averaging lookup table

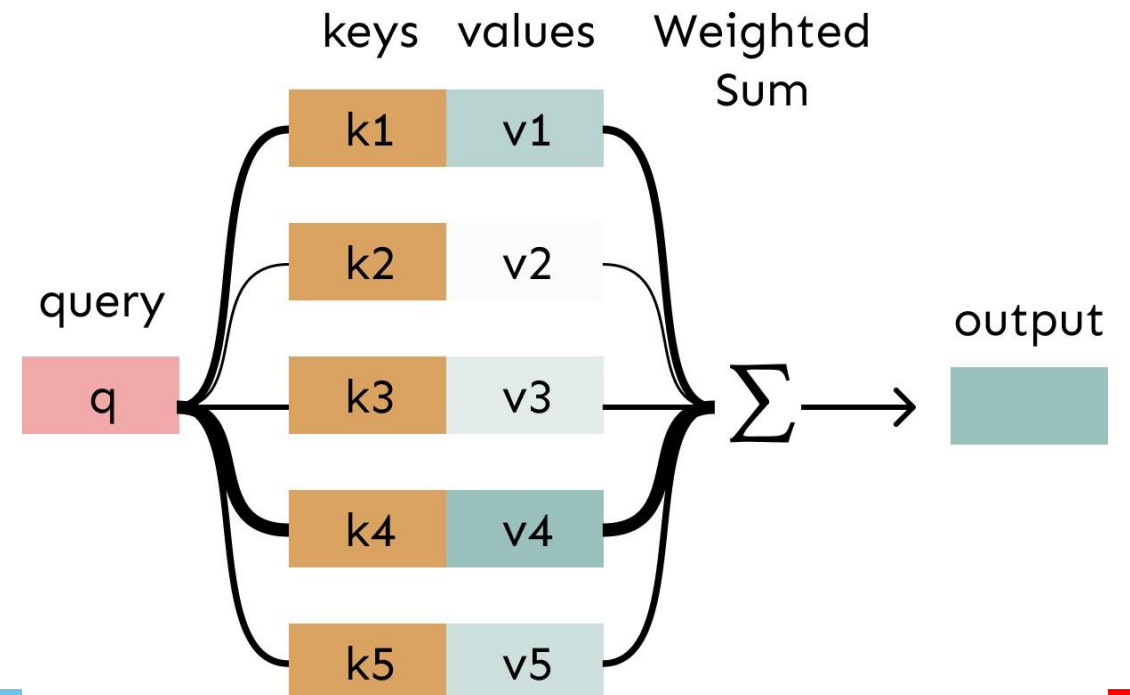


We can think of **attention** as performing fuzzy lookup in a key-value store.

In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



Self-Attention | Transformers



Let us understand how transformers uses self-attention ! One timestamp(one input token)

1. Transform each word embedding with weight matrices W^Q, W^K, W^V , each in $\mathbb{R}^{d \times d}$

$$W^V \quad W^K \quad W^Q \in \mathbb{R}^{d \times d}$$

In Vaswani et al., 2017, d was 1024.

$$\mathbf{q}_i = W^Q \mathbf{x}_i$$

$$\mathbf{k}_i = W^K \mathbf{x}_i$$

$$\mathbf{v}_i = W^V \mathbf{x}_i$$

Query, Q

As the current focus of attention when being compared to all of the other preceding inputs.

Key, K

In its role as a preceding input being compared to the current focus of attention.

Value, V

As a value used to compute the output for the current focus of attention

Three different roles each $\mathbf{x}_i$ (input embedding), in the computation of self attention

Self-Attention | Transformers



$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i \quad \mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i \quad \mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

2. Compute pairwise similarities between keys and queries (alignment score)

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

The simple dot product can be an arbitrarily large;
scaled dot-product is used in transformers

d_k = dimensionality of the query and key
vectors acts as regularization and
improve performance of larger models

3. Normalize with softmax

$$\begin{aligned} \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^i \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \leq i \end{aligned}$$

Alignment scores measures of how well
the query and keys match

4. Compute output for each word as weighted sum of values

$$\mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Self-Attention | Transformers



$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i$$

$$\mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i$$

$$\mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

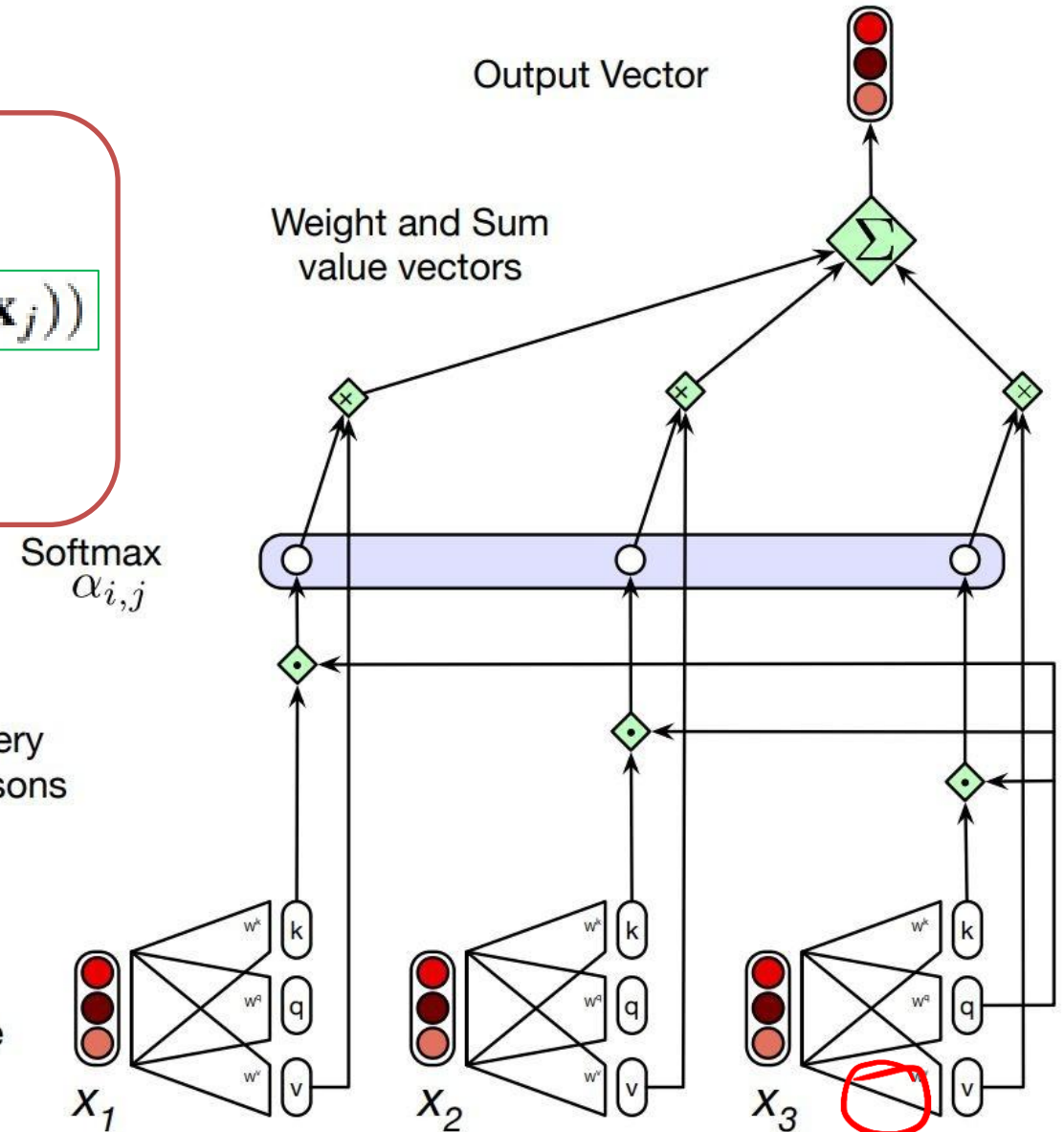
$$\mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

- Each output, y , is computed independently
- Entire process can be parallelize

Calculating the value of y_3 , the third element of a sequence using causal (left-to-right) self-attention

Generate
key, query, value
vectors

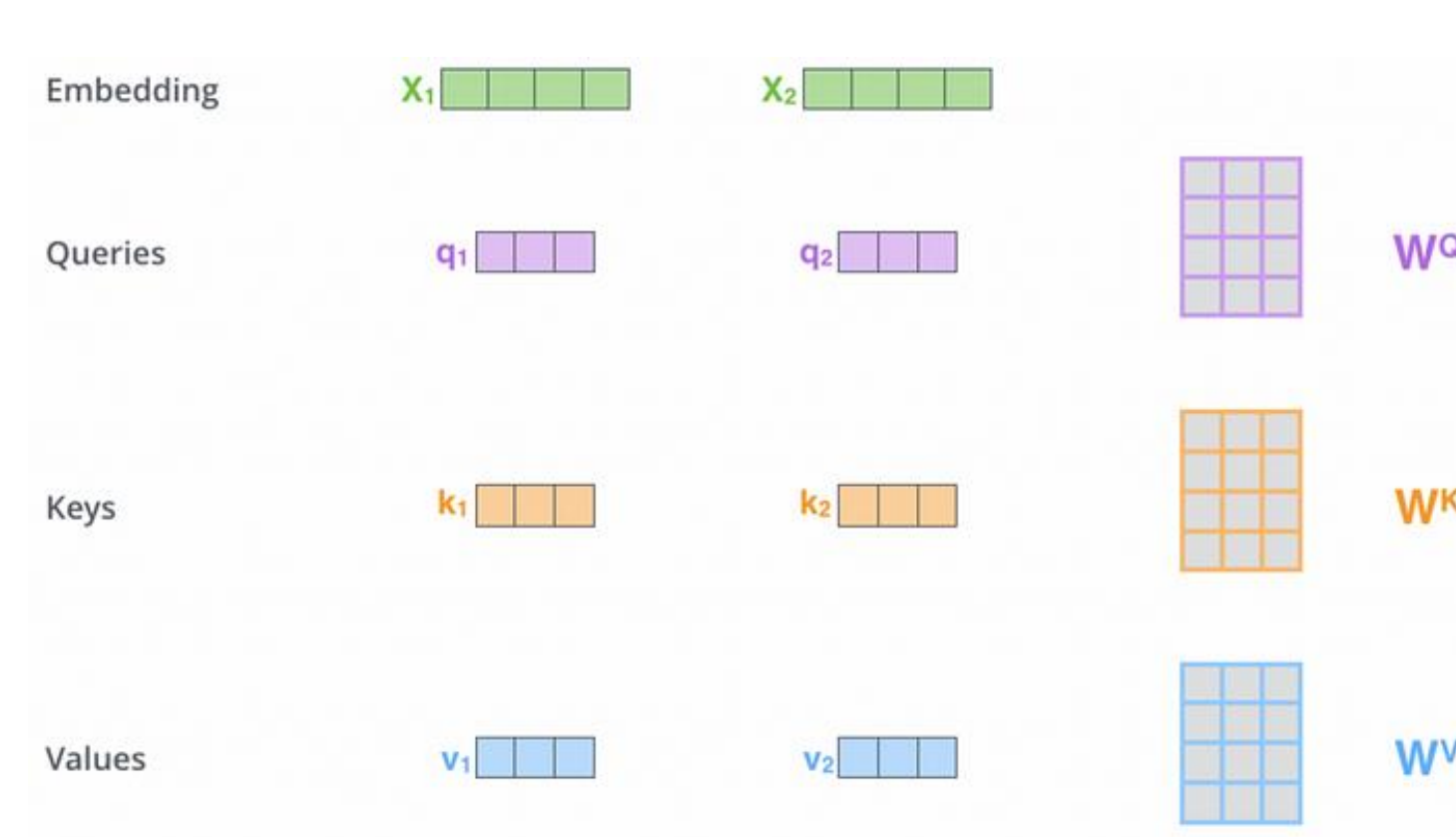
Key/Query
Comparisons



Parallelized using efficient matrix multiplication



Create three vectors from each of the encoder's input values (query, key, value)



We sometimes say that the *query attends to the values*. E.g. in the seq2seq + attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values).

Self-Attention | Transformers

For N tokens



- Pack the input embeddings of the N input tokens into a single matrix

$$\mathbf{X} \in \mathbb{R}^{N \times d}$$

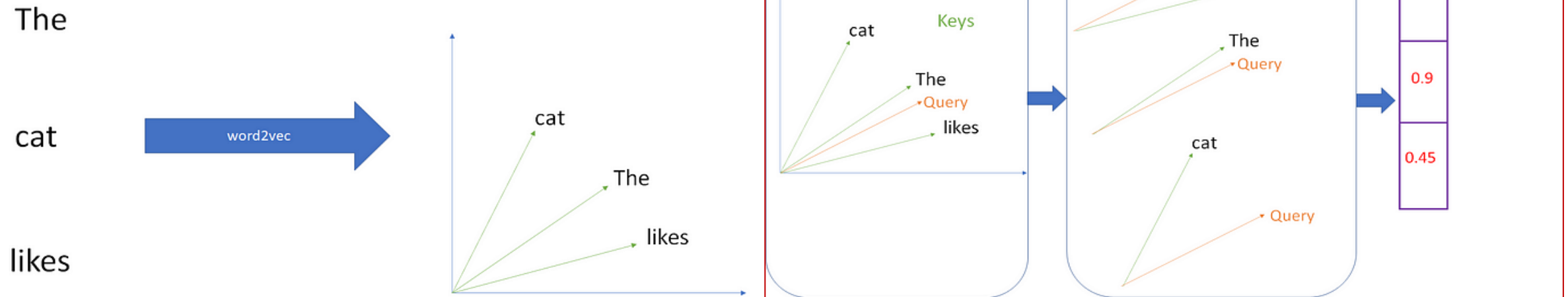
- Each row of X is the embedding of one token of the input

- Multiply X by the key, query, and value (dxd) matrices

$$\mathbf{Q} = \mathbf{XW}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{XW}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{XW}^{\mathbf{V}}$$

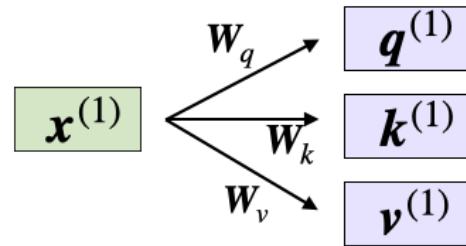
$$\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^{\top}}{\sqrt{d_k}} \right) \mathbf{V}$$

Self-Attention Hypothetical Example



Example (masked future)

Defining the Weight Matrices

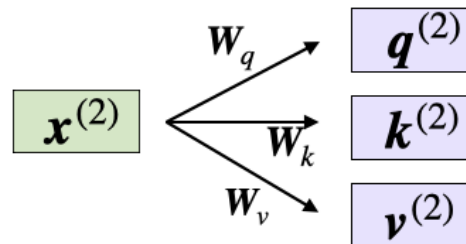


Playing

$q1 = [0.212 \quad 0.04 \quad 0.63 \quad 0.36]$

$k1 = [0.31 \quad 0.84 \quad 0.963 \quad 0.57]$

$v1 = [0.36 \quad 0.83 \quad 0.1 \quad 0.38]$

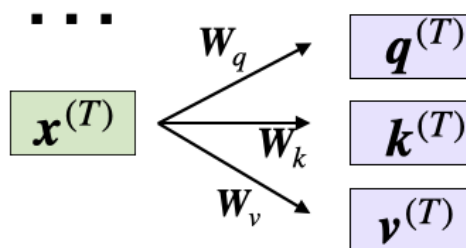


Outside

$q2 = [0.1 \quad 0.14 \quad 0.86 \quad 0.77]$

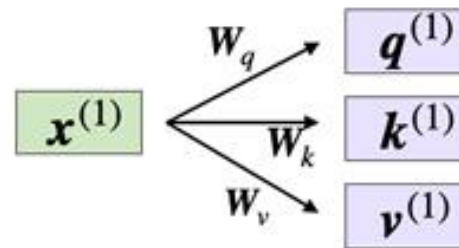
$k2 = [0.45 \quad 0.94 \quad 0.73 \quad 0.58]$

$v2 = [0.31 \quad 0.36 \quad 0.19 \quad 0.72]$



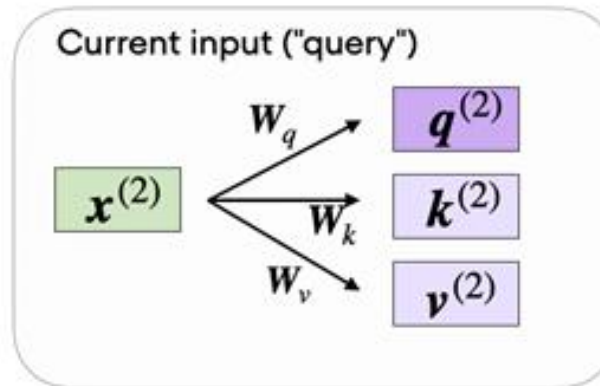
Example

Computing the Unnormalized Attention Weights



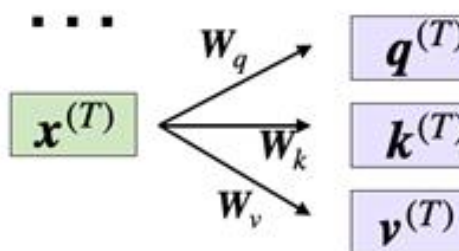
$$q^{(2)\top} \cdot k^{(1)}$$

$$= \text{Score}(X_2, X_1)$$



$$q^{(2)\top} \cdot k^{(2)}$$

$$= \text{Score}(X_2, X_2)$$

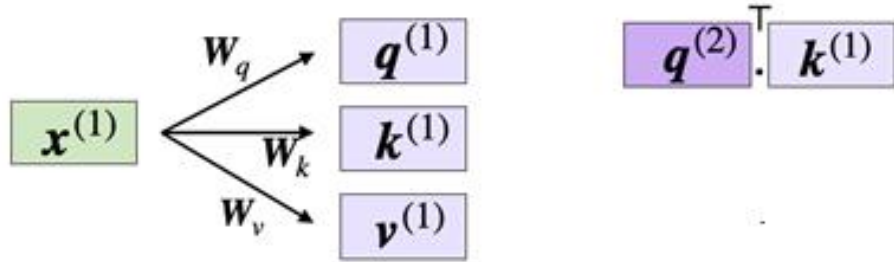


$$q^{(2)\top} \cdot k^{(T)}$$

$$= \text{Score}(X_2, X_T)$$

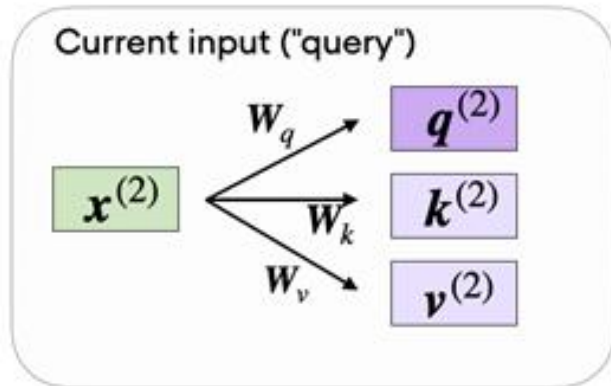
Example

Computing the Attention Scores



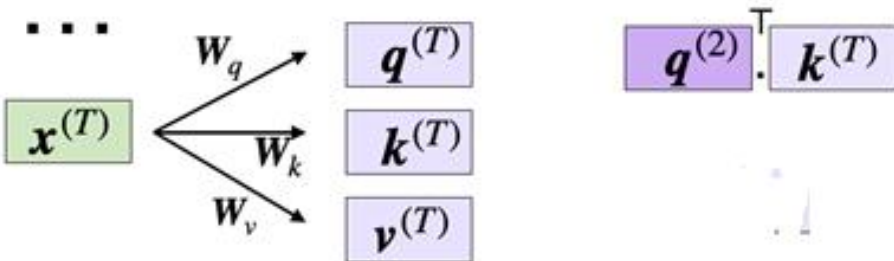
$$= \text{Score}(X_2, X_1)$$

$$\alpha_{21} = \text{softmax} \left(\frac{\text{Score}(X_2, X_1)}{\sqrt{d_k}} \right)$$



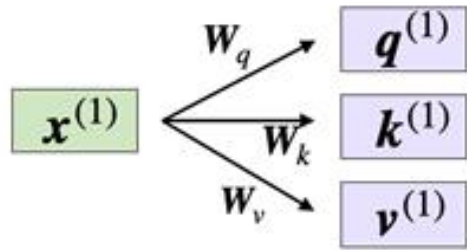
$$= \text{Score}(X_2, X_2)$$

$$\alpha_{22} = \text{softmax} \left(\frac{\text{Score}(X_2, X_2)}{\sqrt{d_k}} \right)$$



Example

Computing the Attention Scores

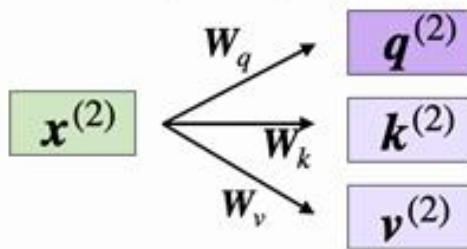


$$q^{(2)\top} \cdot k^{(1)} = \text{Score}(X_2, X_1)$$

$$\alpha_{21} = \text{softmax} \left(\frac{\text{Score}(X_2, X_1)}{\sqrt{d_k}} \right)$$

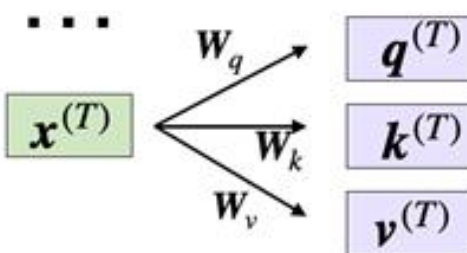
$$y_2 = \sum_{j=1}^2 \alpha_{2j} v_j$$

Current input ("query")



$$q^{(2)\top} \cdot k^{(2)} = \text{Score}(X_2, X_2)$$

$$\alpha_{22} = \text{softmax} \left(\frac{\text{Score}(X_2, X_2)}{\sqrt{d_k}} \right)$$



$$q^{(2)\top} \cdot k^{(T)}$$

Solved example (without masking the future)



Playing
q1 = [0.212 0.04 0.63 0.36]
k1 = [0.31 0.84 0.963 0.57]
v1 = [0.36 0.83 0.1 0.38]
Outside
q2 = [0.1 0.14 0.86 0.77]
k2 = [0.45 0.94 0.73 0.58]
v2 = [0.31 0.36 0.19 0.72]

Playing	Outside
Dot Product q1.k1 = 0.91121 q1.k2 = 0.8017	Dot Product q2.k1 = 1.41568 q2.k2 = 1.251
Scaled Dot Product, w'=4 $e_{11} = 0.455605$ $e_{12} = 0.40085$	Scaled Dot Product, w'=4 $e_{21} = 0.70784$ $e_{22} = 0.6255$

Playing	Outside
$\alpha_{11} = \frac{\exp(e_{11})}{\exp(e_{11}) + \exp(e_{12})} = 0.513685331$	$\alpha_{21} = \frac{\exp(e_{21})}{\exp(e_{21}) + \exp(e_{22})} = 0.520573378$
$\alpha_{12} = \frac{\exp(e_{12})}{\exp(e_{11}) + \exp(e_{12})} = 0.486314669$	$\alpha_{22} = \frac{\exp(e_{22})}{\exp(e_{21}) + \exp(e_{22})} = 0.479426622$

$$y_1 = \alpha_{11} \cdot v_1 + \alpha_{12} \cdot v_2 = [0.335684267 \quad 0.601432106 \quad 0.14376832 \quad 0.545346987]$$

$$y_2 = \alpha_{21} \cdot v_1 + \alpha_{22} \cdot v_2 = [0.336028669 \quad 0.604669487 \quad 0.143148396 \quad 0.543005052]$$

Solved example (without masking the future)



Playing
q1 = [0.212 0.04 0.63 0.36]
k1 = [0.31 0.84 0.963 0.57]
v1 = [0.36 0.83 0.1 0.38]
Outside
q2 = [0.1 0.14 0.86 0.77]
k2 = [0.45 0.94 0.73 0.58]
v2 = [0.31 0.36 0.19 0.72]

Playing	Outside
Dot Product q1.k1 = 0.91121 q1.k2 = 0.8017	Dot Product q2.k1 = 1.41568 q2.k2 = 1.251
Scaled Dot Product, w'=4 $e_{11} = 0.455605$ $e_{12} = 0.40085$	Scaled Dot Product, w'=4 $e_{21} = 0.70784$ $e_{22} = 0.6255$

Playing	Outside
$\alpha_{11} = \frac{\exp(e_{11})}{\exp(e_{11}) + \exp(e_{12})} = 0.513685331$	$\alpha_{21} = \frac{\exp(e_{21})}{\exp(e_{21}) + \exp(e_{22})} = 0.520573378$
$\alpha_{12} = \frac{\exp(e_{12})}{\exp(e_{11}) + \exp(e_{12})} = 0.486314669$	$\alpha_{22} = \frac{\exp(e_{22})}{\exp(e_{21}) + \exp(e_{22})} = 0.479426622$

$$y_1 = \alpha_{11} \cdot v_1 + \alpha_{12} \cdot v_2 = [0.335684267 \quad 0.601432106 \quad 0.14376832 \quad 0.545346987]$$

$$y_2 = \alpha_{21} \cdot v_1 + \alpha_{22} \cdot v_2 = [0.336028669 \quad 0.604669487 \quad 0.143148396 \quad 0.543005052]$$

Barriers and solutions for Self-Attention as a building block

Barriers

- Doesn't have an inherent notion of order!
- No nonlinearities for deep learning magic! It's all just weighted averages
- Need to ensure we don't "look at the future" when predicting a sequence
 - Like in machine translation
 - Or language modeling

Solutions

- • Add position representations to the inputs
- • Easy fix: apply the same feedforward network to each self-attention output.
- • Mask out the future by artificially setting attention weights to 0!



Fixing the first self-attention problem: sequence order



- With RNNs, information about the order of the inputs was built into the structure of the model.
- self-attention ditches sequential operations in favor of parallel computation
- Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.
- Consider representing each **sequence index** as a **vector**
 $\mathbf{p}_i \in \mathbb{R}^d$, for $i \in \{1, 2, \dots, n\}$ are position vectors
- Easy to incorporate this info into our self-attention block: just add the $\mathbf{p}_i$ to our inputs!
- $\mathbf{x}_i$ is the embedding of the word at index i . The positioned embedding is:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

Transformers

Position encoding : simple way

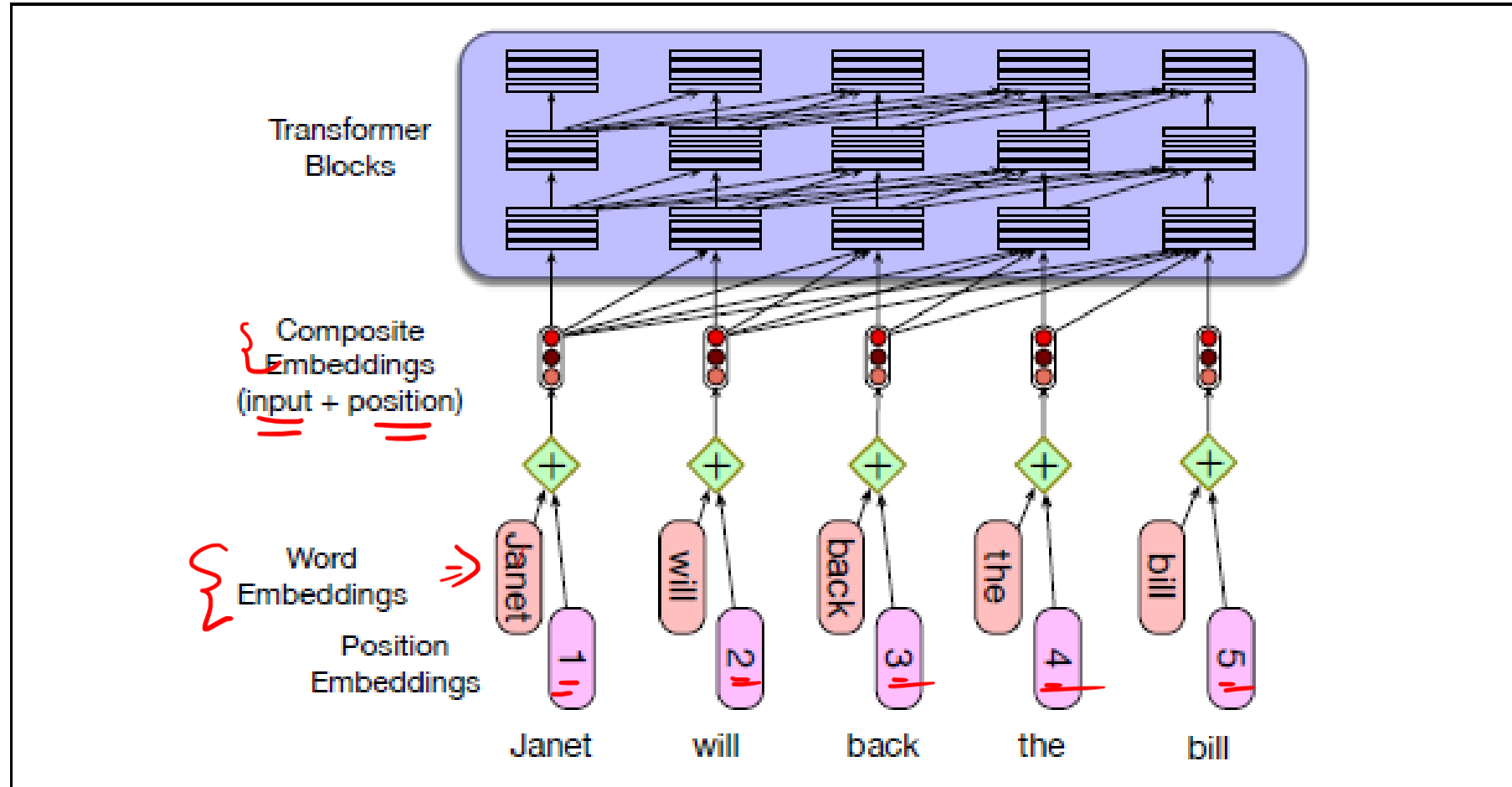


Figure 9.20 A simple way to model position: simply adding an embedding representation of the absolute position to the input word embedding.

Transformers

Position representation vectors through sinusoids



Each position/index is mapped to a vector

Sequence	Index of token, k	Positional Encoding Matrix with d=4, n=100			
		i=0	i=0	i=1	i=1
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

d= Dimension of the output embedding space

i= Used for mapping to column indices $0 \leq i < d/2$, with a single value of i maps to both sine and cosine functions

A combination of sine and cosine functions with differing frequencies was used in the original transformer work.

Transformers

Masking the future in self-attention



We can look at these
(not greyed out) words

$$\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

- Calculation of the comparisons in QK results in a score for each query value to every key value, **including those that follow the query**
- Inappropriate in the setting of language modeling
- To use self-attention in **decoders**, we need to ensure we can't peek at the future.
- To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$

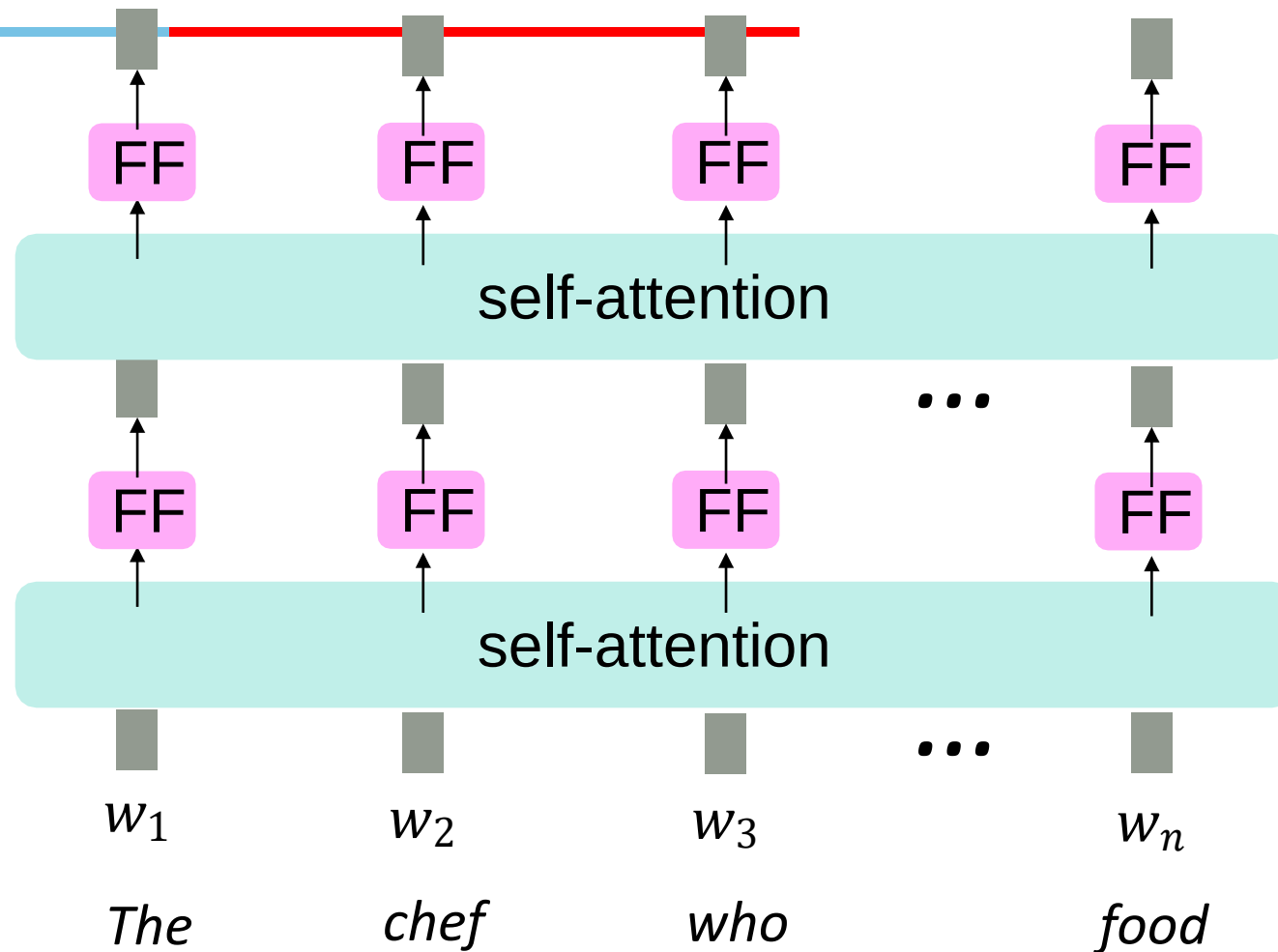
For encoding
these words

	[START]	The	chef	who
[START]		$-\infty$	$-\infty$	$-\infty$
The			$-\infty$	$-\infty$
chef				$-\infty$
who				

Transformers

Adding nonlinearities in self-attention

- Note that there are **no elementwise nonlinearities in self-attention**; stacking more self-attention layers just re-averages **value** vectors
- Easy fix: add a **feed-forward network** to post-process each output vector.



Intuition: the FF network processes the result of attention



Transformers

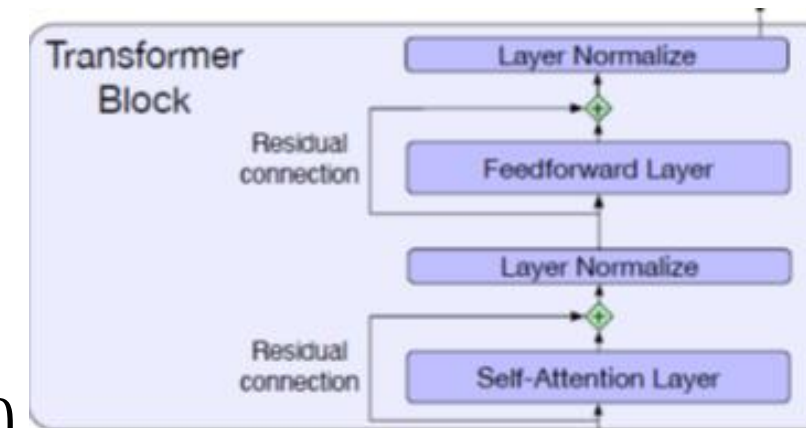
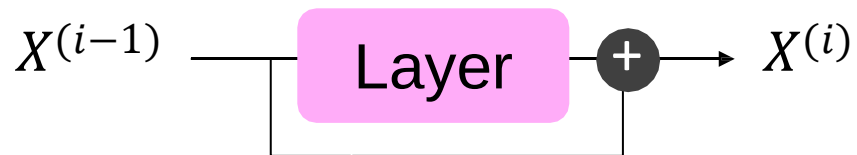
Residual connections

- **Residual connections** are a trick to help models train better.
- Pass information from a lower layer to a higher layer without going through the intermediate layer.
- Allowing information from the activation going forward and the gradient going backwards to skip a layer

Instead of $X^{(i)} = \text{Layer}(X^{(i-1)})$ (where i represents the layer)



We let $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$
(so we only have to learn “the residual” from the previous layer)



Transformers

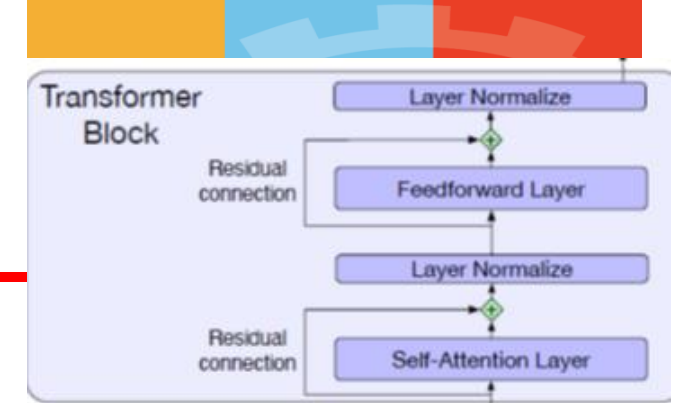
Layer normalization

- **Layer normalization** is a trick to help models train faster.
 - Layer norm is a variation of the standard score, or z-score, from statistics applied to a single hidden layer
- Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model.
- d_h = dimension of word vector

$$\text{LayerNorm} = \gamma \hat{x} + \beta$$

γ and β are learnable parameters

$$\begin{aligned} \mathbf{z} &= \text{LayerNorm}(\mathbf{x} + \text{SelfAttn}(\mathbf{x})) \\ \mathbf{y} &= \text{LayerNorm}(\mathbf{z} + \text{FFNN}(\mathbf{z})) \end{aligned}$$



$$\hat{x} = \frac{(x - \mu)}{\sigma}$$

$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

Layer normalization example



2 words and 3 dimensions

$$\begin{bmatrix} 0.2 & 0.1 & 0.3 \\ 0.5 & 0.1 & 0.1 \end{bmatrix}$$

$$\mu_{11} = \frac{1}{3} [0.2 + 0.1 + 0.3] = 0.2$$

$$\mu_{21} = \frac{1}{3} [0.5 + 0.1 + 0.1] = 0.233$$

$$\sigma_{11} = 0.8164, \sigma_{21} = 0.1885$$

$$\hat{x} = \begin{bmatrix} 0 & -1.2248 & 1.2248 \\ 1.414 & -0.707 & -0.707 \end{bmatrix}$$

$$\hat{x} = \frac{(x - \mu)}{\sigma}$$

$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i$$
$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

Multihead-Attention



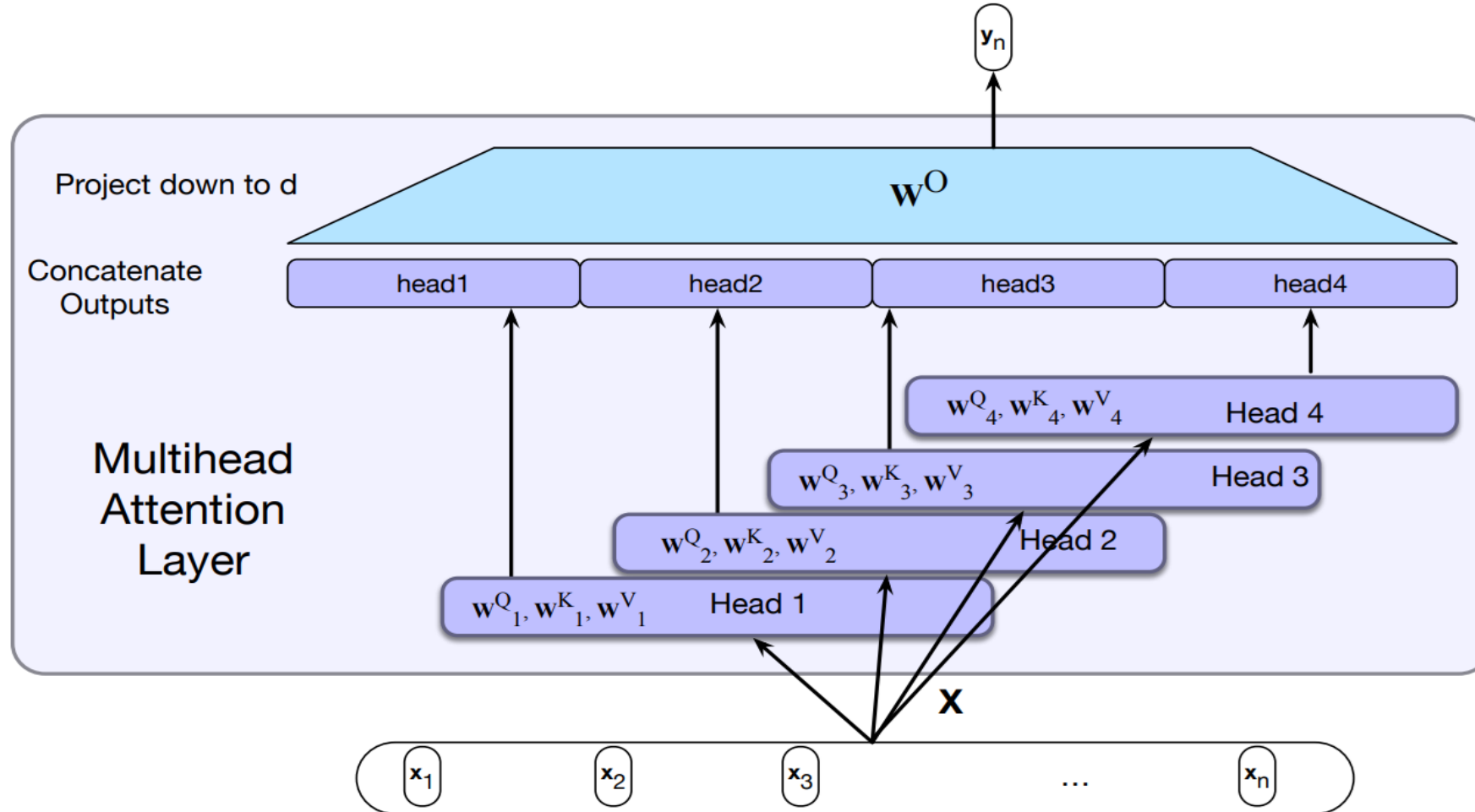
- Different words in a sentence can relate to each other in many different ways simultaneously
 - >> A single transformer block to learn to capture all of the different kinds of parallel relations among its inputs is inadequate.
- **Multihead** self-attention layers
 - >> **Heads** $\Rightarrow$ sets of self-attention layers, that reside in parallel layers at the same depth in a model, each with its own set of parameters.
 - >> Each head learn different aspects of the relationships that exist among inputs at the same level of abstraction

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \dots \oplus \mathbf{head}_h) \mathbf{W}^O$$

$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

Multihead-Attention

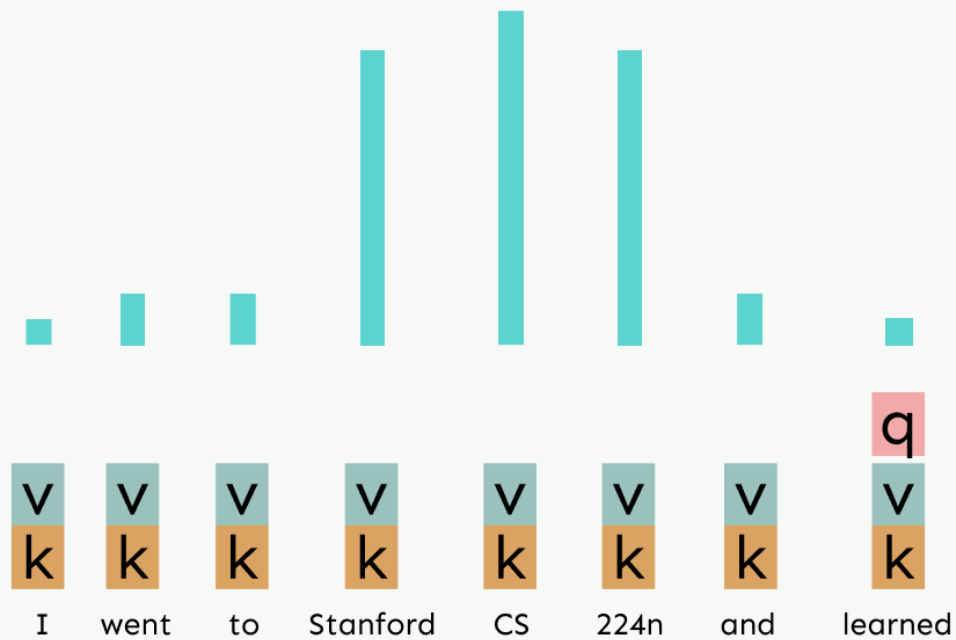


Each of the multihead self-attention layers is provided with its own set of key, query and value weight matrices. The outputs from each of the layers are concatenated and then projected down to d , thus producing an output of the same size as the input so layers can be stacked.

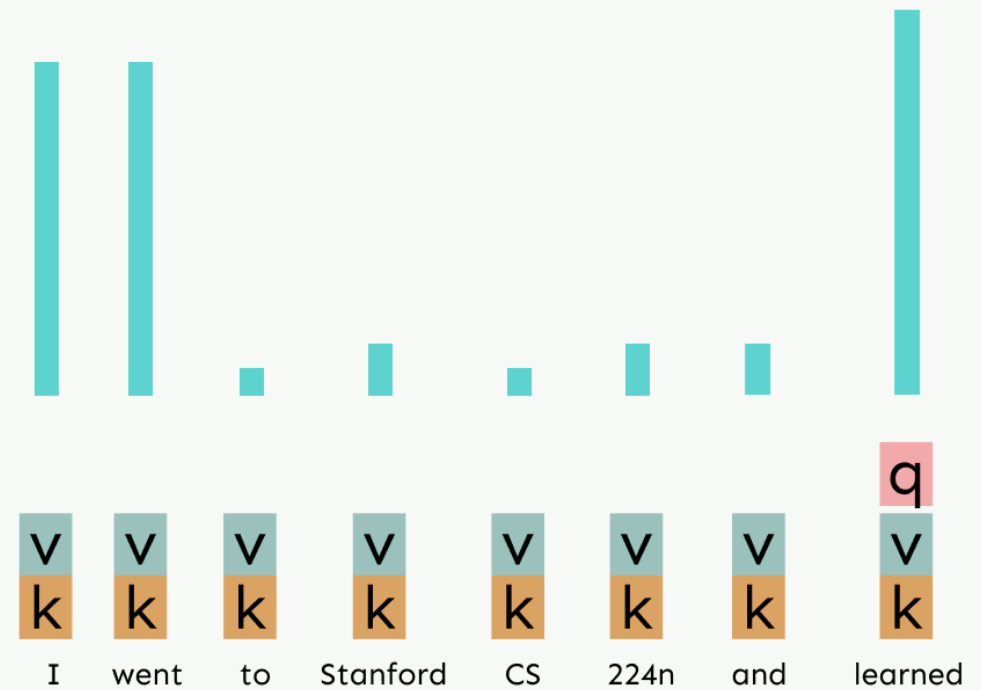
Hypothetical Example of Multi-Head Attention



Attention head 1
attends to entities



Attention head 2 attends to
syntactically relevant words



I went to Stanford CS 224n and learned

Bidirectional self-attention model

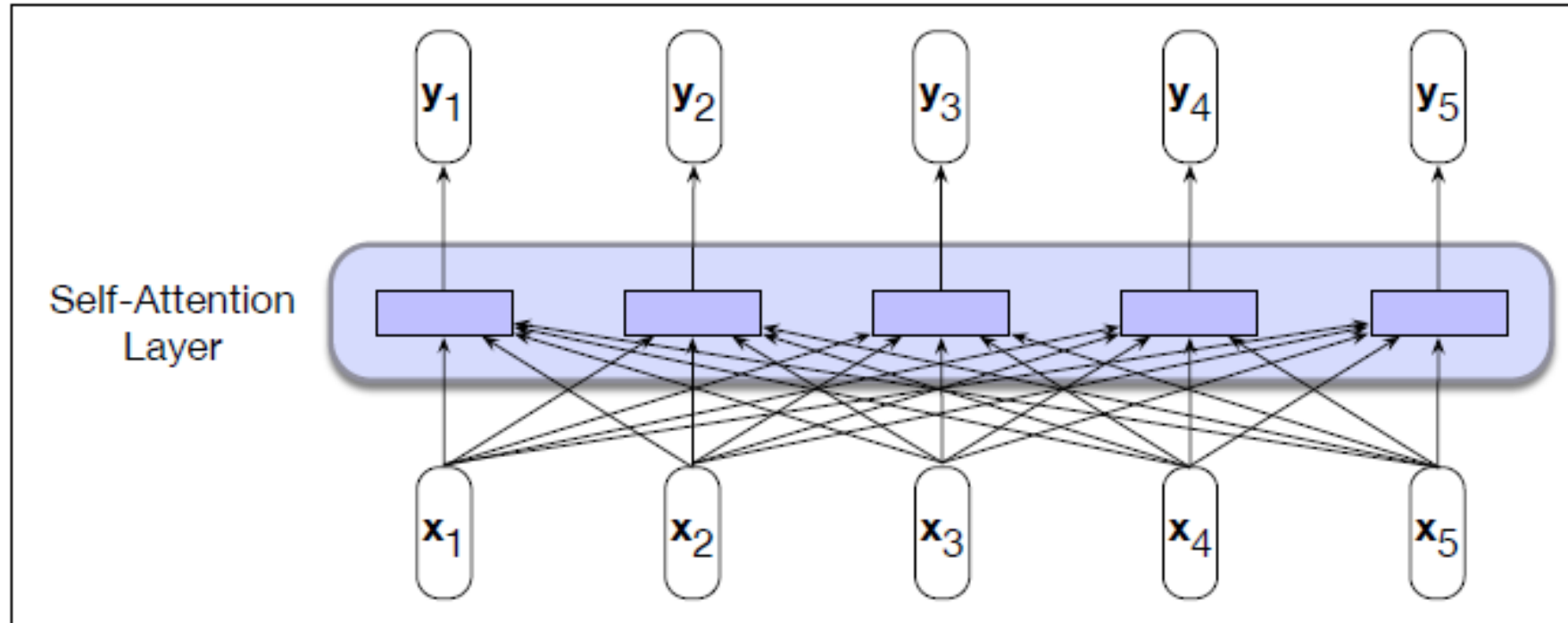


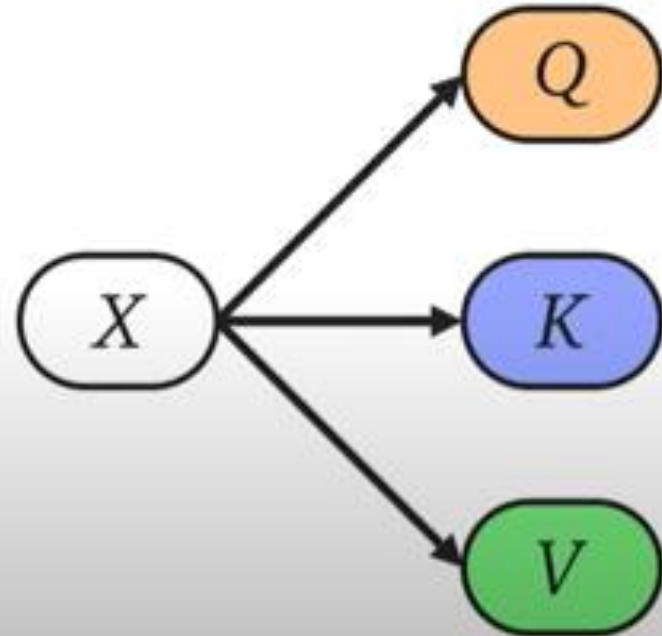
Figure 11.2 Information flow in a bidirectional self-attention model. In processing each element of the sequence, the model attends to all inputs, both before and after the current one.

Self Attention Vs Cross Attention



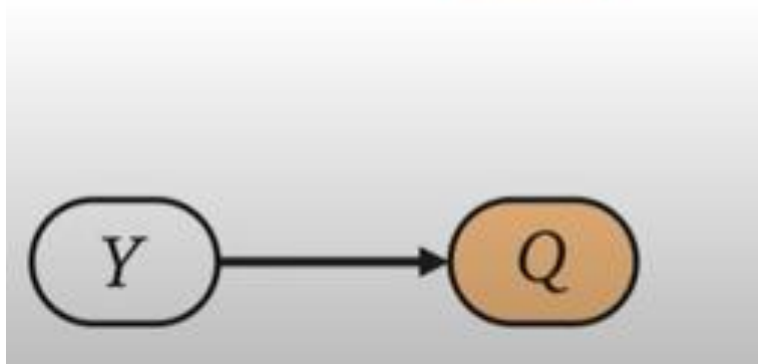
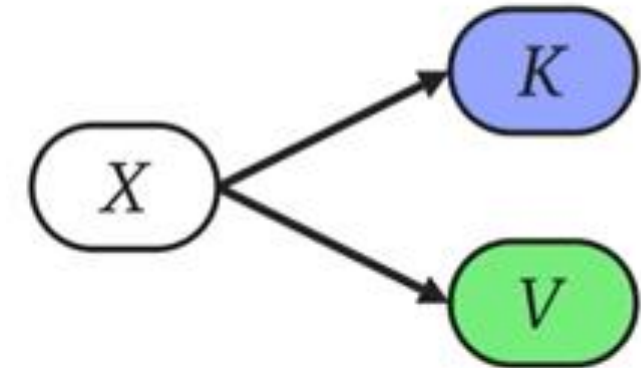
$$Q = XW^Q; K = XW^K; V = XW^V$$

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



$$Q = W^Q H^{dec[i-1]}; K = W^K H^{enc}; V = W^V H^{enc}$$

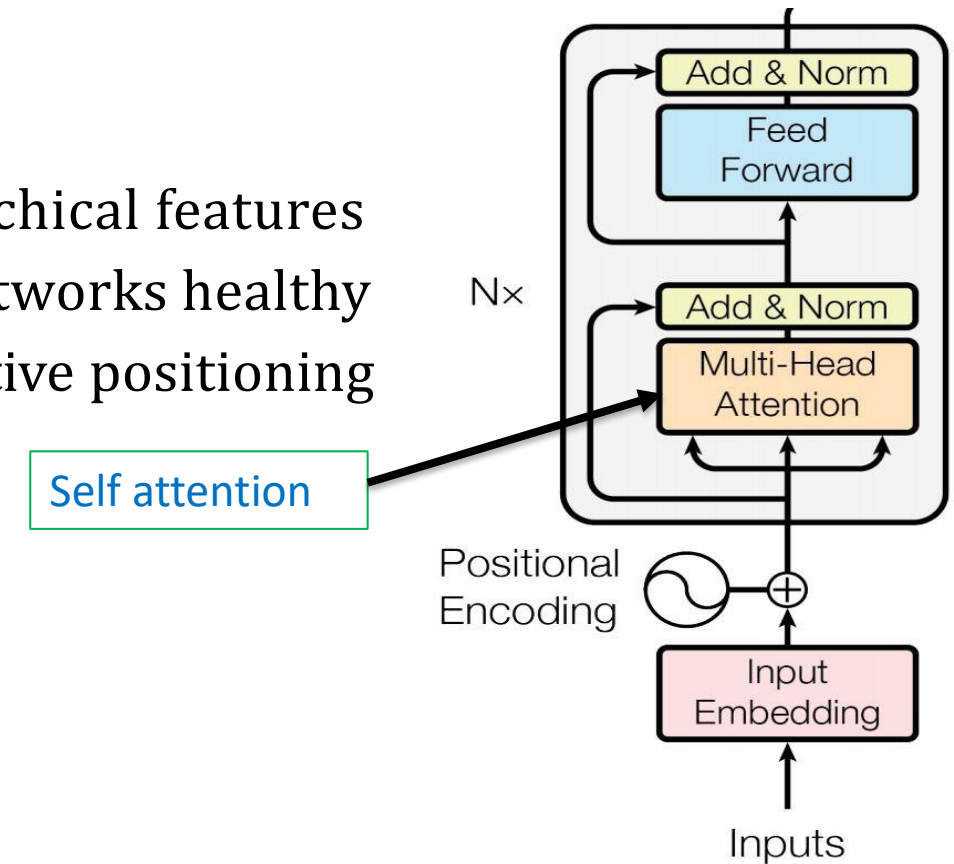
$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



The Transformer Encoder

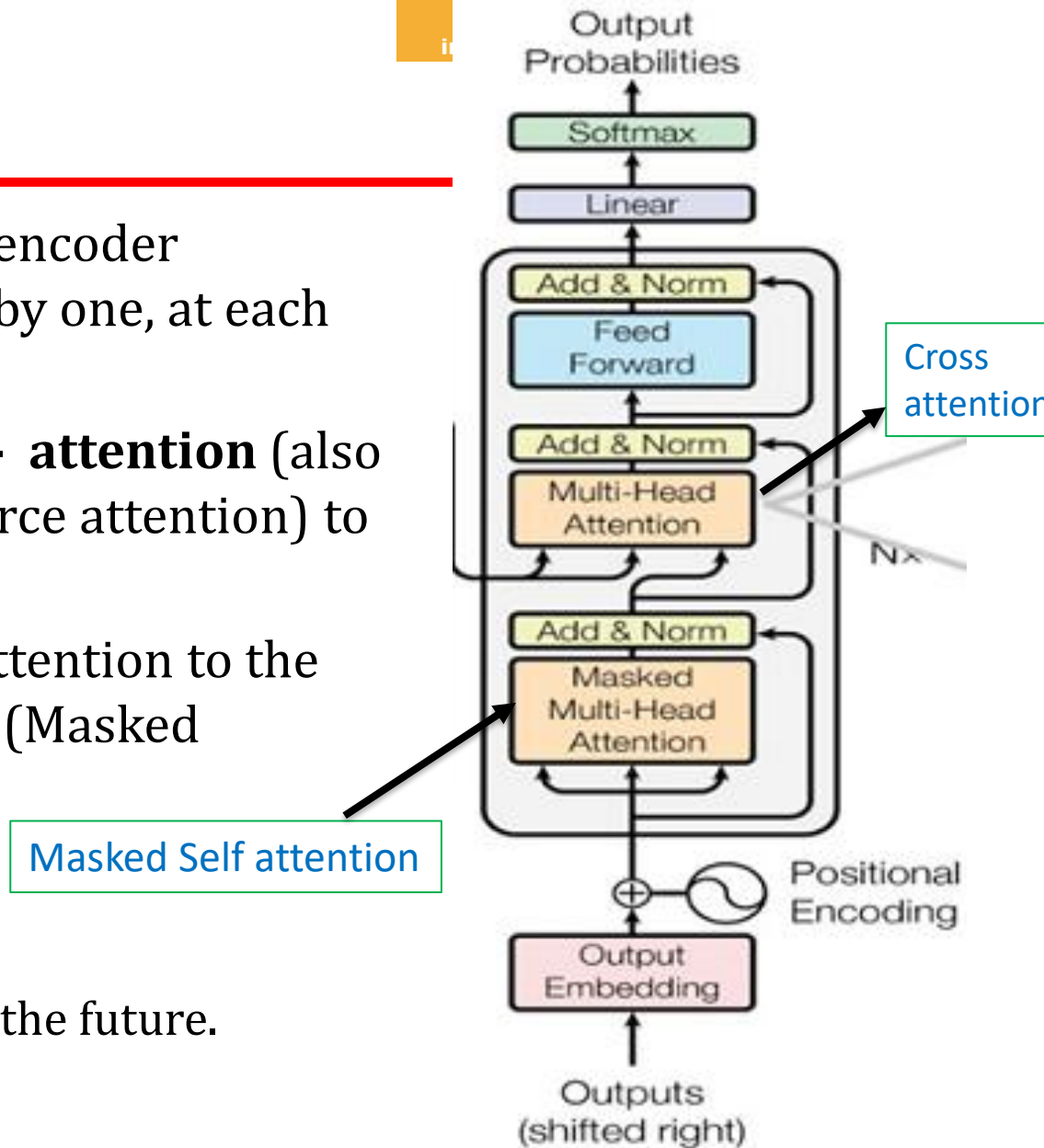


- It receives an input and **builds a representation of it (its features): contextual embeddings.**
- **Bidirectional** model
- A stack of identical layers
- Multi-headed self attention : Models context
- Feed-forward layers :Computes non-linear hierarchical features
- Layer norm and residuals : Makes training deep networks healthy
- Positional embeddings : Allows model to learn relative positioning

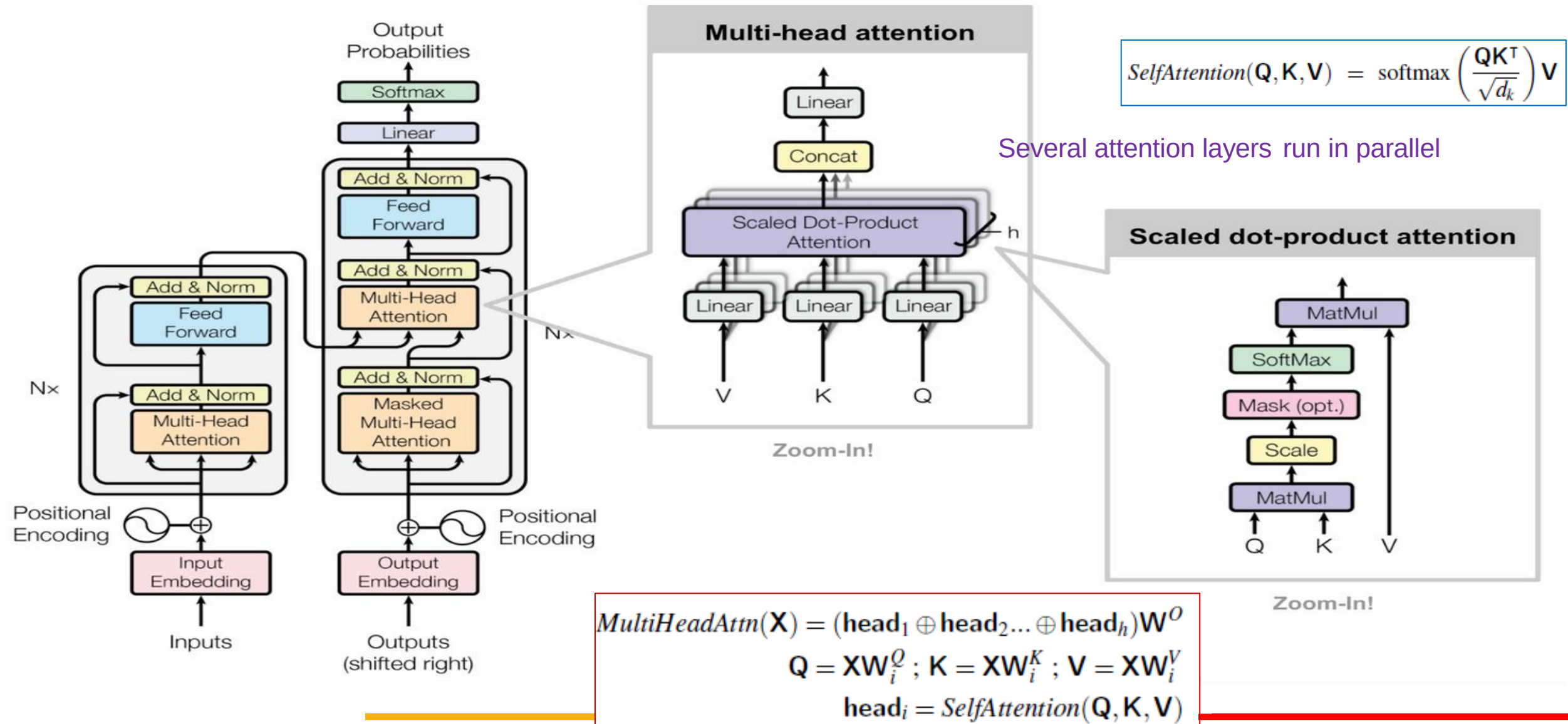


The Transformer Decoder

- **A conditional language model** that attends to the encoder representation and generates the target words one by one, at each timestep
- Transformer Decoder is modified to perform **cross-attention** (also sometimes called encoder-decoder attention or source attention) to the output of the Encoder
- The decoder, works sequentially and can only pay attention to the words in the sentence that it has already translated (Masked attention layer)
- **Unidirectional model**
- **Masking:**
 - In order to parallelize operations while not looking at the future.
 - Keeps information about the future from “leaking” to the past.



The Transformer Encoder-Decoder



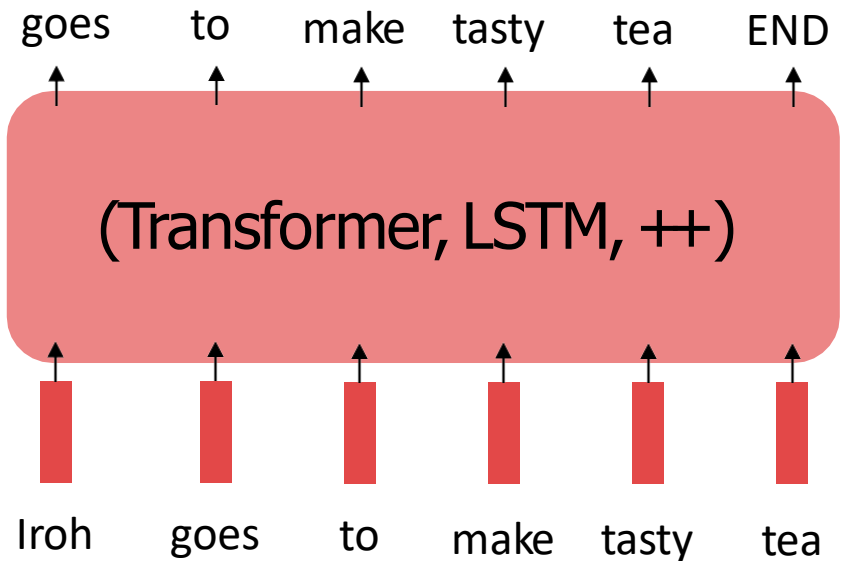
The Pre training / Fine tuning Paradigm



Pretraining can improve NLP applications by serving as parameter initialization.

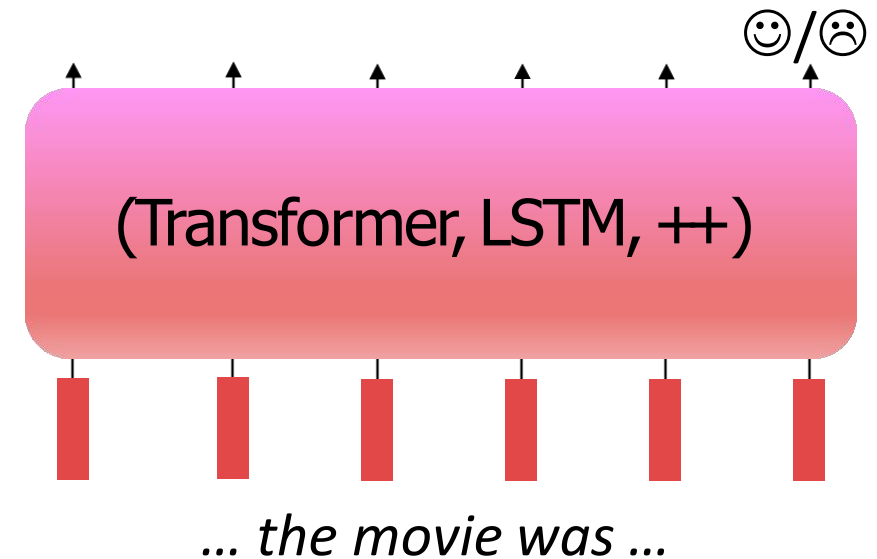
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!

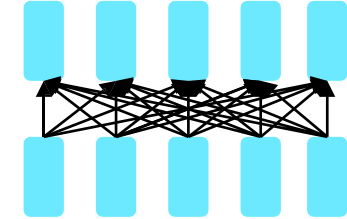


Pre-training for three types of architectures



Encoder-only models

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?
- Also known as *auto-encoding models*
- For tasks that require understanding of the input, e.g sentence classification and named entity recognition
- Representatives of this family of models include:
 - ALBERT, BERT, DistilBERT, , ELECTRA, RoBERTa

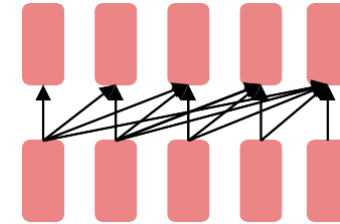


Pre-training for three types of architectures



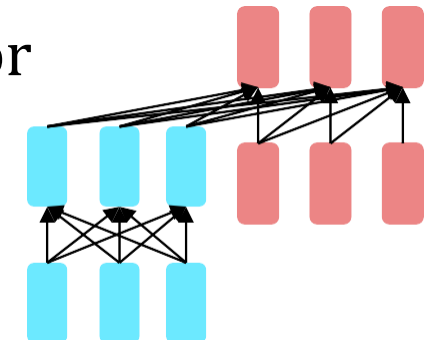
Decoder-only models

- Language models
- Nice to generate from; can't condition on future words
- These models are also known as *auto-regressive models*.
- Representatives of this family of models include:
 - CTRL, GPT series, Transformer XL



Encoder-decoder models or sequence-to-sequence models

- For generative tasks that require an input, such as translation or summarization.
- BART/T5-like



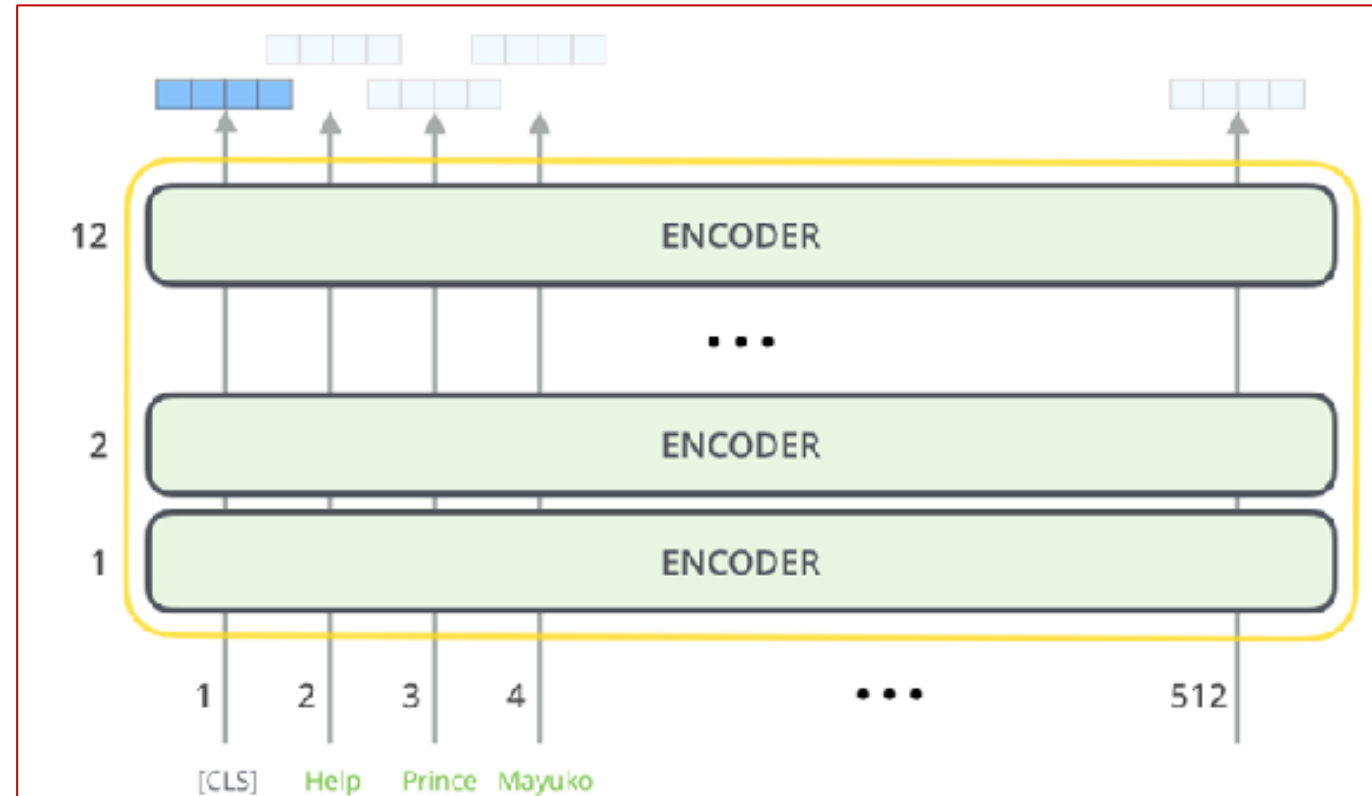
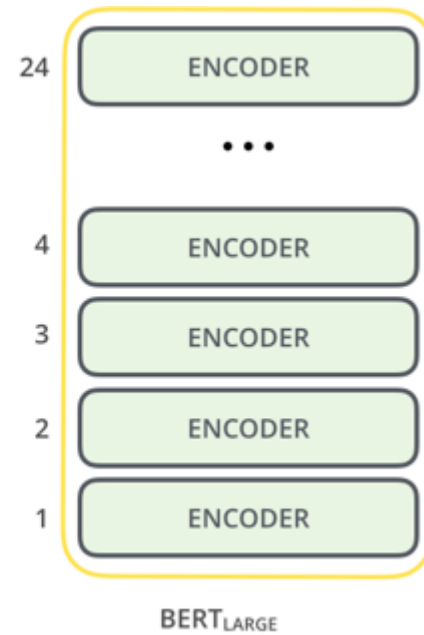
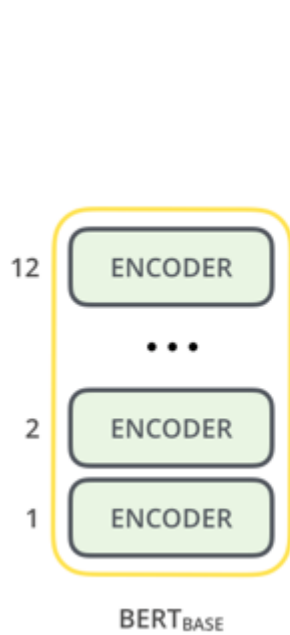
Transformers and NLP



BERT: Bidirectional Encoder Representations from Transformers

- Two models were released:
 - BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
 - BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
 - BooksCorpus (800 million words)
 - English Wikipedia (2,500 million words)
- Pre training is expensive and impractical on a single GPU.
 - BERT was pre trained with 64 TPU chips for a total of 4 days.
 - (TPUs are special tensor operation acceleration hardware)
- Fine tuning is practical and common on a single GPU
 - “Pre train once, fine tune many times.”

BERT



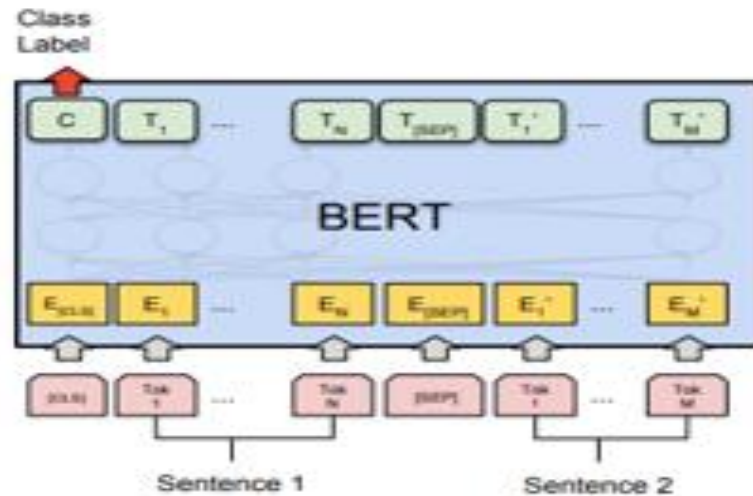
Model input dimension 512

BERT for different task

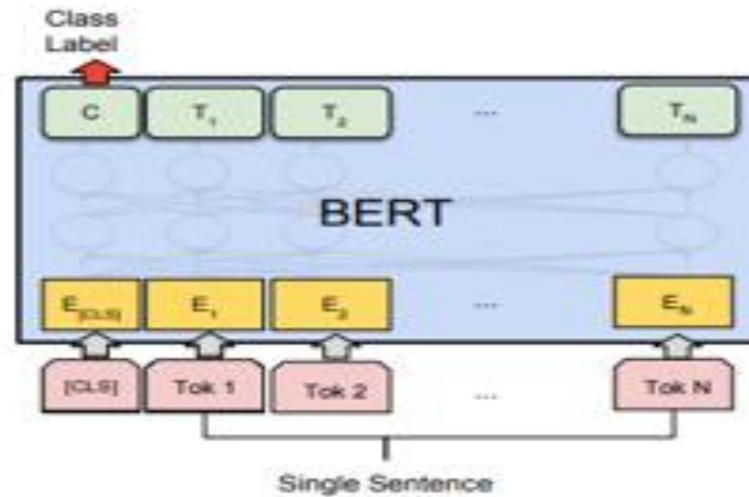
innovate

achieve

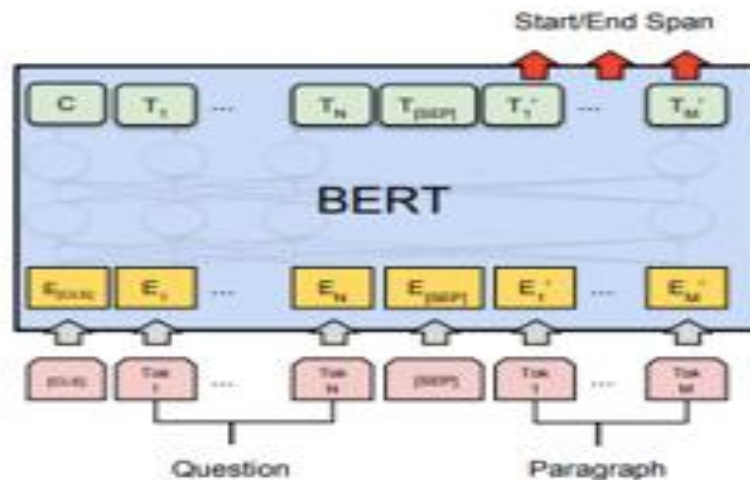
lead



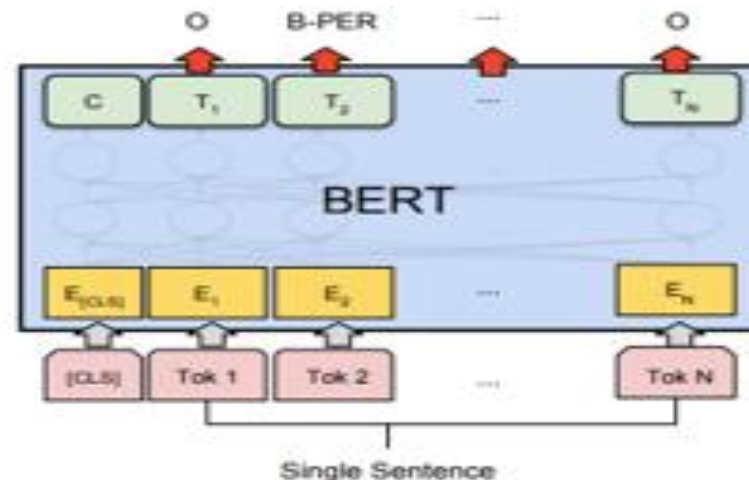
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Transformers and Computer Vision

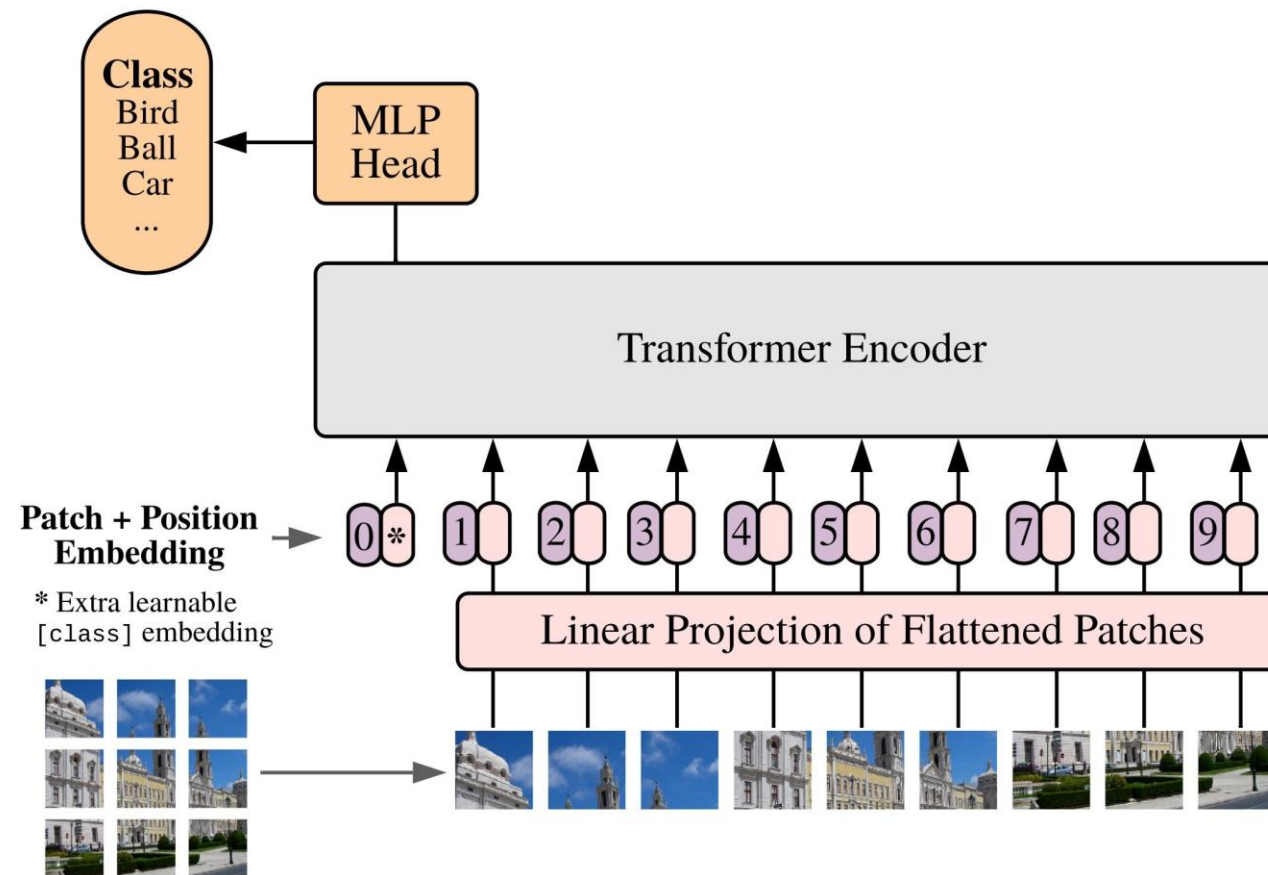


- CNNs are the architecture of choice in Vision
- Transformers are the architecture of choice in NLP
- Numerous attempts to incorporate self-attention into CNNs:
 - [Wang CVPR 2018](#), [Bello ICCV 2019](#), [Huang ICCV 2019](#), [Carion ECCV 2020](#)
- Or to replace convolutions entirely with self-attention
 - [Parmar ICML 2018](#), [Ramachandran NeurIPS 2019](#)

Vision Transformers



An Image is Worth 16x16 Words



Vision Transformer Models



Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

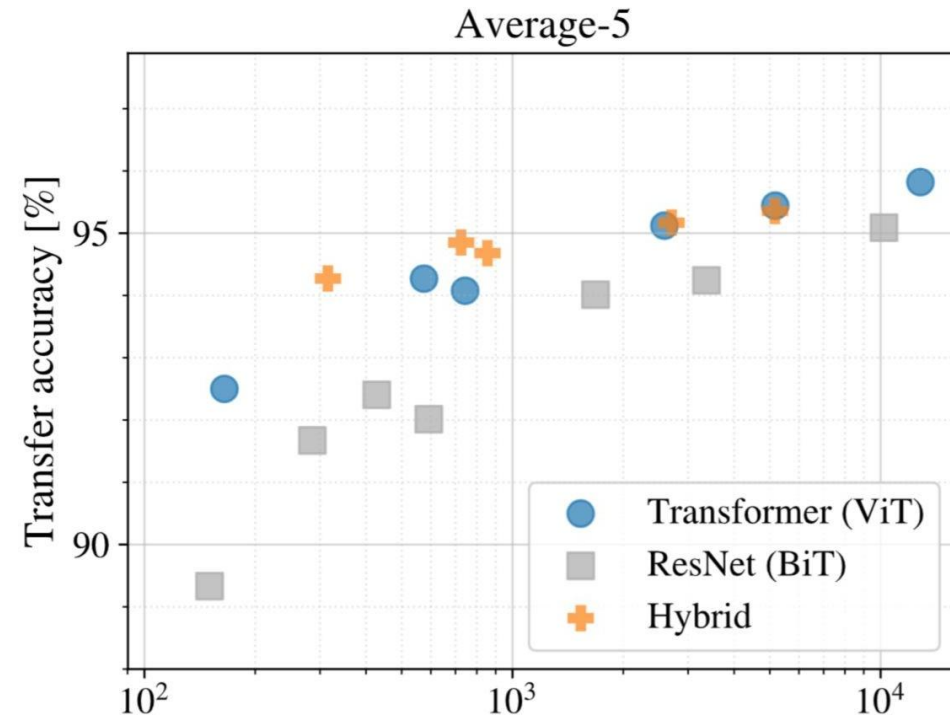
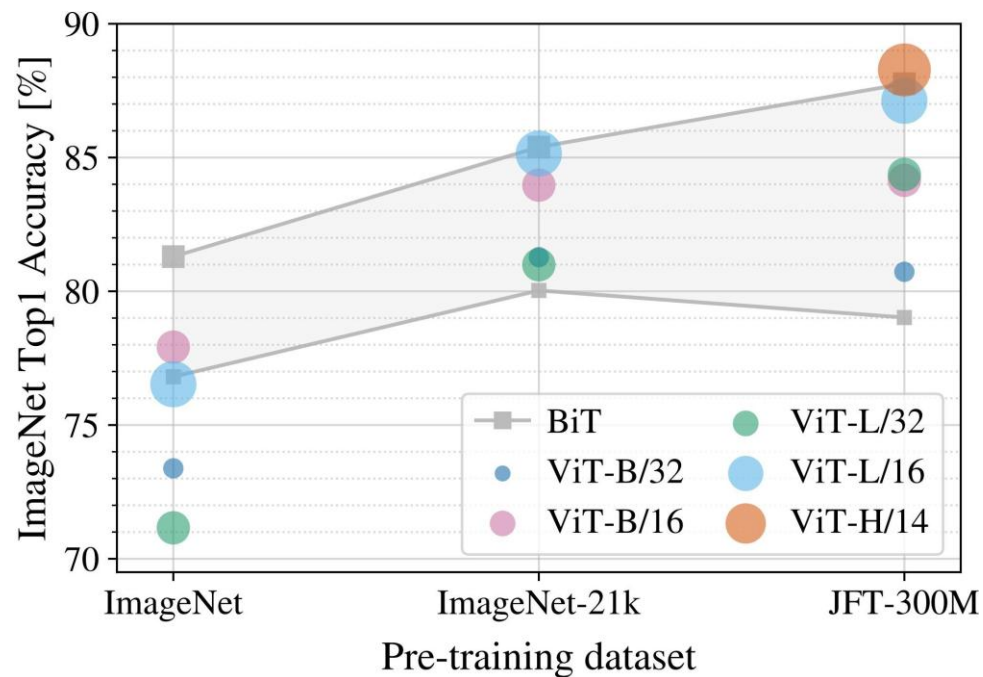


Notation e.g. ViT-L/16

Vision Transformers are effective at scale



- Transformers have less inductive biases than Convolutional Networks (ie translational equivariance) So they need more data to train
- Transformers, are however, able to take advantage of large-scale data better than CNNs can And are more compute-efficient too in terms of computation to reach accuracy.



- Speech and Language Processing by Daniel Jurafsky
- <https://jalammar.github.io/illustrated-bert/>
- <https://huggingface.co/course/chapter1/4?fw=pt>
- <https://arxiv.org/abs/1706.03762>
- <https://arxiv.org/abs/1810.04805>
- <https://arxiv.org/abs/1406.1078>
- <https://arxiv.org/abs/1609.08144>